Chapter 17 — Interview Question Bank

Last reviewed: October 2026. Model names, prices and protocol versions change quickly; where an answer depends on them, it says so and points to the dated section of the book.

This chapter collects the GenAI and LLM questions that show up again and again in interviews for AI engineer, ML engineer, applied scientist and "GenAI developer" roles. The wording follows the way the questions appear in public question lists, GitHub interview collections and candidates' interview write-ups (see the sources at the end). Chapter 15's 52-question bank is a compressed revision of this book; this bank is wider and follows the interviewer's wording instead of the book's order.

How to use this bank

- **Questions are worded the way they are usually asked.** Interviewers rarely say "explain §4.6"; they say "What is RLHF and why do we need it?". Practise against that wording.
- **★ marks a question asked very often** — it appeared in several independent question lists or interview reports. If you are short of time, drill only the ★ questions first.
- **Each answer is something you can say out loud** in 20–60 seconds: 2–6 sentences, or a short code snippet with one or two sentences of explanation. Every term is defined the first time it appears.
- **→ *More:*** links to the section of this book that teaches the topic in full (with code and real output). A few questions have no link because the book does not cover them elsewhere; their answer here is complete on its own.
- **How to drill:** cover the answer, say yours aloud (or write it in under a minute), then uncover and check. Mark the ones you missed and come back to them the next day. Chapter 15's `code/drill.py` shuffles the Chapter 15 bank; the same "cover, say, check" rhythm works here.
- **Interviewers follow up.** For each ★ question, prepare one follow-up: a trade-off ("what does it cost?"), a failure mode ("when does it break?") or a number ("how big is it?").

Group	Questions
1. GenAI and LLM fundamentals	Q1–Q12
2. Tokenization and embeddings	Q13–Q24
3. Transformers and attention	Q25–Q42
4. Pre-training, post-training and fine-tuning (SFT, RLHF, DPO, LoRA)	Q43–Q62
5. Decoding, context windows and hallucination	Q63–Q77
6. Prompt engineering	Q78–Q91
7. Building LLM applications (APIs, structured output, tools, memory)	Q92–Q102
8. Retrieval-augmented generation (RAG)	Q103–Q120
9. Vector databases and similarity search	Q121–Q130
10. Evaluation (LLM-as-judge, benchmarks, RAG metrics)	Q131–Q144
11. Agents, tool calling and MCP	Q145–Q158
12. Safety, guardrails and prompt injection	Q159–Q170
13. Inference serving, quantization, cost and latency	Q171–Q190
14. Diffusion and generative models beyond text	Q191–Q199
15. GenAI system design and scenario questions	Q200–Q215

1. GenAI and LLM fundamentals

Q1. ⋆ What is the difference between predictive (discriminative) AI and generative AI? A discriminative model learns to separate classes — it estimates P(label | input), for example "is this email spam?". A generative model learns the distribution of the data itself, so it can produce new samples: text, images, audio or code. LLMs are generative: they model the probability of the next token given the previous ones and sample from it. In practice generative models are also used for classification (ask the LLM for a label), which is often the quickest way to a first version. → *More:* §1.5 The AI family tree, §4.1 What an LLM is

Q2. ⋆ What is a Large Language Model (LLM), and how does it work? An LLM is a neural network (almost always a decoder-only transformer) with billions of parameters, trained to predict the next **token** (a piece of a word) over trillions of tokens of text. At inference it repeats one step: read the tokens so far, output a probability for every token in its vocabulary, pick one, append it, repeat. Everything it "knows" is stored in its weights from training; anything else must be put into the prompt. "Large" refers to both parameter count and training data. → *More:* §4.1 What an LLM is, §4.2 Next-token prediction

Q3. ⋆ How are LLMs trained? Walk me through the stages.

1. **Pre-training**: self-supervised next-token prediction on a huge web/code/book corpus — produces a *base model* that continues text but does not follow instructions well.
2. **Supervised fine-tuning (SFT)**: train on curated (instruction, good response) pairs so it behaves like an assistant.
3. **Preference tuning**: RLHF (a reward model plus reinforcement learning) or DPO (a direct loss on preferred vs rejected answers) to make outputs more helpful and safe.
4. Often **reinforcement learning on verifiable rewards** (maths answers, passing unit tests) to build reasoning skill. Pre-training is by far the most expensive stage; the later stages shape behaviour. → *More:* §4.3 Pretraining, §4.5 SFT, §4.6 RLHF and DPO

Q4. What is the difference between a base model and an instruct (chat) model? A base model is the raw pre-trained model: given "The capital of France is", it continues the text, and given a question it may answer with more questions. An instruct model has been fine-tuned (SFT plus preference tuning) to follow instructions, use a chat template with system/user/assistant roles, refuse harmful requests and stop at a sensible point. Use instruct models for products; base models are mainly a starting point for your own fine-tuning or research. → *More:* §4.5 SFT

Q5. What are the main differences between LLMs and traditional statistical language models (n-grams)? An n-gram model counts how often each word follows the previous n−1 words, so it sees only a short, fixed window and cannot generalise to word combinations it never counted. An LLM learns dense representations (embeddings) and uses attention over the whole context window, so it captures long-range dependencies and similarities between words it has seen in different contexts. LLMs also transfer: one model does translation, summarisation and coding through prompting, whereas n-gram models are trained per task. → *More:* §4.1 How LLMs differ from earlier NLP

Q6. What is self-supervised learning, and why is it so important for LLM pre-training? Self-supervised learning creates labels from the data itself: in next-token prediction, every position in a document is a training example whose label is the next token. No human labelling is needed, so the training set can be the whole public web, which is what makes the scale of LLMs possible. Humans only label the much smaller post-training sets. → *More:* §4.2 Next-token prediction

Q7. ⋆ What are scaling laws? What did the Chinchilla paper show? Scaling laws are empirical power-law curves: test loss falls smoothly and predictably as you increase parameters, data and compute. Kaplan et al. (2020) first measured them; Hoffmann et al. (2022, "Chinchilla") found that for a fixed compute budget you should grow parameters and tokens together — roughly **20 training tokens per parameter** — and that many earlier models were too big for their data. Today teams often "over-train" smaller models far past 20 tokens per parameter because a smaller model is cheaper to serve to millions of users. → *More:* §4.4 Scaling laws

Q8. What are emergent abilities, and are they real? "Emergent abilities" are skills (multi-step arithmetic, following complex instructions) that seemed to appear suddenly once models passed a certain

size. Later work showed much of the suddenness comes from the metric: with all-or-nothing scoring (exact match) progress looks like a jump, while smoother metrics show steady improvement. A good answer: capabilities do improve with scale, but "sudden emergence" is partly a measurement artefact, so do not plan on unpredictable leaps. → *More:* §4.4 Emergent abilities (with a caveat)

Q9. What is a Mixture of Experts (MoE) model? An MoE layer replaces the single feed-forward network in each transformer block with many "expert" feed-forward networks and a small **router** that sends each token to only a few of them (for example 2 of 64). The model therefore has a very large total parameter count but uses only a fraction of it per token, so it gets more capacity at roughly the compute cost of a smaller dense model. The costs: all experts must still sit in memory, routing needs load balancing during training, and serving across GPUs is more complex. → *More:* §3.6 The transformer block

Q10. What are reasoning models, and when would you use one? Reasoning models (OpenAI's o-series, DeepSeek-R1, Claude with extended thinking, Gemini thinking models) are trained with reinforcement learning to produce a long internal chain of thought before the final answer. They do much better on maths, coding, planning and multi-step analysis, but they cost more output tokens and add seconds to minutes of latency. Use them for hard, high-value problems; use a fast standard model for simple extraction, classification or chat. → *More:* §4.11 Reasoning models

Q11. ⋆ Open-weight vs closed (API) models: how do you choose? Closed API models (Claude, GPT, Gemini) give the best quality with no infrastructure work, but you pay per token, send data to a provider and depend on their roadmap. Open-weight models (Llama, Mistral, Qwen, DeepSeek, Gemma) can be self-hosted for data control, fine-tuned freely and become cheaper at high, steady volume — but you run GPUs, serving, scaling and safety yourself. The deciding variables: data residency and privacy, volume (cost per token at your scale), quality needed, and team skills. Many products use both: an API model for hard requests and a small open model for cheap, high-volume ones. → *More:* §4.13 Open vs closed models, §13.6 Build vs buy

Q12. What are multimodal LLMs and how are images fed into them? A multimodal LLM accepts more than text — images, audio, sometimes video. The common design: an image encoder (often a vision transformer, a transformer that reads an image as a grid of patches) turns the image into a sequence of vectors, a small projection layer maps them into the LLM's embedding space, and the LLM attends to them like extra tokens. Images therefore consume context tokens and cost money just like text. → *More:* §4.12 Multimodal LLMs, §10.8 Multimodal models

2. Tokenization and embeddings

Q13. ⋆ What is a token, and what is tokenization? Why is it necessary? A token is the unit an LLM reads and writes — usually a subword piece such as "un", "believ", "able"; in English one token is roughly 4 characters or ¾ of a word. **Tokenization** splits text into these pieces and maps each to an integer ID from a fixed vocabulary, because neural networks need numbers, not strings. It matters to engineers because context limits, latency and price are all counted in tokens, and odd behaviour (letter counting, arithmetic on long numbers, higher cost for some languages) often traces back to how text was split. → *More:* §3.1 Tokens and tokenization

Q14. Why is subword tokenization preferred over word-level or character-level tokenization? Word-level vocabularies are huge and still miss new words (typos, names, code identifiers), which become an "unknown" token. Character-level vocabularies are tiny but make sequences very long, so attention costs more and the model must learn spelling from scratch. Subword tokenization is the middle ground: frequent words are single tokens, rare words split into known pieces, and nothing is ever out of vocabulary. → *More:* §3.1 Tokens and tokenization

Q15. ⋆ Explain Byte-Pair Encoding (BPE). How does it work? BPE starts from single characters (or bytes) and repeatedly merges the most frequent adjacent pair in the training corpus into a new token, recording each merge, until the vocabulary reaches the target size (for example 100K–200K). To tokenize new text, apply the learned merges in the same order. **Byte-level BPE** (GPT-2 onward) starts from the 256 byte values, so any string — emoji, any script, binary junk — can be encoded.

```python
from collections import Counter
def most_frequent_pair(words):            # words: list of token lists
    pairs = Counter()
    for w in words:
        pairs.update(zip(w, w[1:]))
    return pairs.most_common(1)[0][0]
def merge(words, pair):
    a, b = pair; out = []
    for w in words:
        i, new = 0, []
        while i < len(w):
            if i < len(w) - 1 and (w[i], w[i+1]) == pair: new.append(a + b); i += 2
            else: new.append(w[i]); i += 1
        out.append(new)
    return out
```

→ *More:* §3.1 How it works — BPE

Q16. What is the difference between BPE, WordPiece and SentencePiece? **BPE** merges the most frequent pair. **WordPiece** (BERT) also merges pairs, but picks the merge that most increases the likelihood of the training data, and marks word-continuation pieces with "##". **SentencePiece** is a library rather than an algorithm: it treats the input as a raw character stream including spaces (shown as "▁"), so it needs no language-specific pre-splitting, and it can train either BPE or a **unigram** model (start with a big vocabulary and prune pieces that matter least). Llama 1–2 and T5 used SentencePiece; GPT models use byte-level BPE (tiktoken), and Llama 3 switched to a tiktoken-based byte-level BPE as well. → *More:* §3.1 Real tokenizer families

Q17. Explain the trade-offs of a large vocabulary in LLMs. A larger vocabulary means fewer tokens per text (shorter sequences, cheaper attention, more text fits in the context, better coverage of non-English languages and code). The costs: the embedding and output (softmax) layers grow linearly with vocabulary size, rare tokens get few training examples, and the final softmax over 200K entries costs compute at every step. Modern models have moved from ~32K (Llama 2) to ~128K–260K vocabularies because multilingual efficiency outweighed the cost. → *More:* §3.1 Trade-offs

Q18. How do LLMs handle out-of-vocabulary (OOV) words? With byte-level subword tokenization there are no truly OOV words: an unseen word is split into smaller known pieces, down to single bytes if needed. The model then has to compose meaning from those pieces, which works well for morphology ("un-believ-able") and less well for long random strings, which cost many tokens. → *More:* §3.1 Tokens and tokenization

Q19. Why do LLMs struggle to count the letters in "strawberry" or reverse a string? Because they never see letters — they see token IDs such as "str" + "awberry". Facts about the letters inside a token are only learned indirectly. Arithmetic on long numbers has the same root cause, since numbers are split into irregular chunks. Fixes: ask the model to spell the word out first (so each letter becomes a token), or give it a code tool. → *More:* §3.1 Common mistakes

Q20. ⋆ What are embeddings? How are tokens converted into embeddings in a transformer? An embedding is a dense vector of numbers (hundreds to thousands of dimensions) learned so that items with similar meaning end up close together. In a transformer the first layer is an **embedding matrix** of shape vocabulary × d_model: each token ID simply selects one row — a lookup, equivalent to multiplying a one-hot vector by the matrix. Position information is then added (or applied inside attention with RoPE), and the vectors flow through the transformer blocks, becoming more context-dependent at each layer. → *More:* §3.2 Embeddings

Q21. What is the difference between static and contextual embeddings? Static embeddings (word2vec, GloVe) give each word one fixed vector, so "bank" has the same vector in "river bank" and "bank account". Contextual embeddings (BERT and every transformer layer after the first) depend on the surrounding words, so the two "bank"s get different vectors. Sentence-embedding models used in RAG produce one contextual vector for a whole passage, usually by pooling (averaging) the token vectors of a fine-tuned encoder. → *More:* §3.2 Static vs contextual embeddings

Q22. ⋆ Cosine similarity vs dot product vs Euclidean distance — which should you use for embeddings? **Cosine similarity** compares directions only (the angle between vectors) and ignores length; **dot product** also rewards longer vectors; **Euclidean (L2) distance** is the straight-line distance. If vectors are

normalised to length 1, all three give the same ranking, which is why many systems normalise and use the dot product (fastest). Rule: use the metric the embedding model was trained with — its documentation says which.

```python
import numpy as np
def cosine(a, b):
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))
```

→ *More:* §3.2 Measuring similarity, §7.2 Embeddings: meaning as geometry

Q23. How do you choose an embedding model for a RAG system? How would you benchmark it on your data? Start from a public leaderboard such as **MTEB** (Massive Text Embedding Benchmark) to shortlist a few models, then check practical limits: maximum input length, dimension (storage cost), languages, price or hosting, and licence. The deciding test is your own data: build 100–500 real queries with their known relevant chunks, embed the corpus with each candidate and measure **recall@k** (how often a relevant chunk is in the top k) and MRR. Remember that changing the model later means re-embedding the whole corpus. → *More:* §7.2 Embeddings: meaning as geometry

Q24. Your embedding model's retrieval accuracy is low on your domain. How would you improve it? In rough order of effort: clean and re-chunk the documents (bad chunks are the most common cause); add metadata filters; add keyword search (BM25) alongside vectors (hybrid search) for exact terms such as product codes; add a cross-encoder reranker; rewrite or expand queries; try a stronger or domain-specific model; and finally **fine-tune the embedding model** with contrastive learning on (query, relevant passage) pairs from your domain, using hard negatives (passages that look similar but are wrong). Measure recall@k after each change. → *More:* §7.7 Hybrid search and reranking

3. Transformers and attention

Q25. ⋆ Explain the transformer architecture. A transformer turns a sequence of token embeddings into a sequence of context-aware vectors using a stack of identical **blocks**. Each block has two sub-layers: **multi-head self-attention** (each token gathers information from other tokens) and a position-wise **feed-forward network** (FFN, a small two-layer network applied to each token separately), each wrapped with a **residual connection** (add the input back to the output) and **layer normalisation**. Position information is added because attention ignores order. A decoder-only LLM adds a causal mask and a final linear layer + softmax that turns the last vector into next-token probabilities. → *More:* §3.6 The transformer block, §3.9 Putting it together

Q26. ⋆ Explain self-attention step by step. What are Query, Key and Value? For each token, three vectors are computed by multiplying its embedding with learned matrices: a **query** (what this token is looking for), a **key** (what it offers to be matched on) and a **value** (the information it passes on). Score each query against every key with a dot product, divide by √d_k, apply softmax to turn scores into weights that sum to 1, and take the weighted sum of the values. The output for each token is therefore a mix of information from the tokens most relevant to it. It is "self"-attention because queries, keys and values all come from the same sequence.

```python
import numpy as np
def attention(Q, K, V, causal=True):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)                    # (T, T)
    if causal:
        scores += np.triu(np.full(scores.shape, -np.inf), k=1)
    w = np.exp(scores - scores.max(-1, keepdims=True))
    w /= w.sum(-1, keepdims=True)                      # softmax per row
    return w @ V
```

→ *More:* §3.4 Self-attention, step by step

Q27. ⋆ Why do we divide by √d_k in scaled dot-product attention? If the components of q and k are independent with mean 0 and variance 1, their dot product (a sum of d_k such products) has variance d_k, so scores grow with the head dimension. Large scores push the softmax into a saturated region where one weight is ≈1 and the rest ≈0, and its gradients become tiny, which slows or stalls training. Dividing by √d_k brings the variance back to about 1. → *More:* §3.4 Self-attention

Q28. ⋆ Why use multi-head attention instead of a single head? Several smaller attention heads run in parallel, each with its own Q/K/V projections, so different heads can specialise in different relationships (syntax, coreference, nearby tokens, a delimiter). Their outputs are concatenated and mixed by an output matrix W_O back to d_model. The total compute is about the same as one big head, because each head works in d_model / h dimensions. → *More:* §3.5 Multi-head attention

Q29. What are MQA and GQA, and why do they matter for inference? Multi-query attention (MQA) shares one key/value head across all query heads; **grouped-query attention (GQA)** shares one K/V head per group of query heads (for example 8 K/V heads for 64 query heads). Both shrink the KV cache (the stored keys and values of earlier tokens) by the same factor, so more users fit on a GPU and each generated token reads less memory. GQA keeps nearly full quality and is the default in most current open models. → *More:* §3.5 Production variants

Q30. ⋆ What is the computational complexity of self-attention? What makes transformers heavy on memory? For sequence length T and width d, attention costs **$O(T^2 \cdot d)$** time and $O(T^2)$ memory for the score matrix, per head and layer — doubling the context quadruples that part. At inference, the KV cache grows linearly with T and batch size and often limits concurrency more than compute. Mitigations: FlashAttention (exact attention without storing the T×T matrix), sliding-window or sparse attention, GQA/MQA, quantised KV caches and PagedAttention. → *More:* §3.4 Cost, §11.4 The KV cache

Q31. ⋆ What are positional encodings and why do transformers need them (when CNNs and RNNs don't)? Attention treats the input as a set: shuffle the tokens and each token's output is the same (it is *permutation-invariant*), so "dog bites man" and "man bites dog" would look alike. RNNs process tokens in order and CNNs use local windows, so order is built in; transformers must add it. Options: fixed **sinusoidal** vectors (original 2017 paper), **learned** absolute position vectors (BERT, GPT-2), or relative schemes — **RoPE** (rotary position embedding, rotates queries and keys by a position-dependent angle; used by Llama, Qwen and most modern LLMs) and **ALiBi** (adds a distance penalty to scores). → *More:* §3.3 Positional encoding

Q32. How can you extend the context length of an LLM beyond what it was trained on? With RoPE models, rescale positions so longer sequences map into the trained range — **position interpolation**, **NTK-aware scaling** or **YaRN** — then fine-tune briefly on long documents. Add efficient attention (FlashAttention, sliding-window layers) and memory savers (GQA, KV-cache quantisation) so the longer context fits. Always test long-context recall afterwards ("needle in a haystack" plus harder multi-fact tests), because accepting long input is not the same as using it well. → *More:* §3.3 Positional encoding, §4.8 Context windows

Q33. ⋆ What is masked (causal) self-attention, and why does the decoder need it? A causal mask sets the attention scores for future positions to −∞ before the softmax, so token t can only attend to tokens 1…t. During training the whole sequence is processed at once, and without the mask the model could simply read the answer (the next token) instead of predicting it. The mask makes training match generation, where future tokens do not exist yet. → *More:* §3.7 Causal masking

Q34. What is cross-attention, and how does it differ from self-attention? In cross-attention the queries come from one sequence (the decoder's tokens) and the keys and values from another (the encoder's output), so the decoder can look at the source sentence while generating — the classic use is translation. In self-attention Q, K and V all come from the same sequence. Text-to-image diffusion models use cross-attention so image features can attend to the text prompt. → *More:* §3.8 Encoder vs decoder, §10.5 Text-to-image

Q35. ⋆ Encoder-only vs decoder-only vs encoder-decoder: which for what? Encoder-only (BERT): bidirectional attention, best for understanding tasks — classification, named-entity recognition, embeddings and rerankers. **Decoder-only** (GPT, Claude, Llama): causal attention, generates left to right — chat, writing, code; it won for general LLMs because one simple objective scales well and every task can be phrased as text generation. **Encoder-decoder** (T5, BART, Whisper): reads a whole input then generates an output — translation, summarisation, speech-to-text. → *More:* §3.8 Encoder vs decoder vs encoder-decoder

Q36. What is the difference between causal language modelling and masked language modelling? Causal LM (GPT): predict the next token from the previous ones only — natural for generation. **Masked LM** (BERT): hide about 15% of tokens and predict them from both sides — gives stronger representations for

understanding but cannot generate fluent text left to right. → *More:* §3.8 Encoder vs decoder, §4.2 Next-token prediction

Q37. What is the purpose of the feed-forward network in each transformer block? Attention mixes information *between* tokens; the FFN then processes each token *on its own* with a two-layer network that expands the width (typically 4× d_model, or a gated variant such as SwiGLU) and projects back. It holds most of the block's parameters (about two-thirds) and is where much factual knowledge appears to be stored. MoE models replace it with routed experts. → *More:* §3.6 The three ingredients

Q38. Why do transformers use residual connections and layer normalisation? Residual connections add each sub-layer's input to its output (x + f(x)), giving gradients a direct path through dozens of layers — this is how transformers avoid vanishing gradients — and letting each layer learn a small correction. **Layer normalisation** rescales each token's vector to a stable mean and variance, keeping activations in range. Modern LLMs use **pre-norm** (normalise before each sub-layer), often with RMSNorm, because it trains more stably than the original post-norm. → *More:* §3.6 The transformer block

Q39. What is softmax, and where is it used in a transformer? Softmax turns a vector of real numbers (logits) into probabilities: exponentiate each and divide by the sum. It appears twice: inside attention (turning scores into weights) and at the output (turning the final logits over the vocabulary into next-token probabilities). In code, subtract the maximum first to avoid overflow. → *More:* §3.4 Self-attention

Q40. ⋆ Compare transformers and RNNs/LSTMs. Why did transformers win? RNNs process tokens one after another, so training cannot be parallelised across the sequence and information from far back must survive many steps (vanishing gradients, limited memory). Transformers connect every pair of tokens directly in one step and process the whole sequence in parallel on GPUs, which made training on huge datasets practical. The price is quadratic attention cost and a KV cache that grows with context, whereas an RNN keeps a fixed-size state — which is why state-space models (Mamba) and hybrids are still researched. → *More:* §3.9 Transformers vs RNNs in one table

Q41. What is FlashAttention? FlashAttention (Dao et al., 2022) computes exact attention — the same result — but in tiles that fit in the GPU's fast on-chip memory (SRAM), never writing the full T×T score matrix to slow GPU memory (HBM). Because attention is limited by memory traffic rather than arithmetic, this gives large speed-ups and memory that grows linearly with sequence length. It is a kernel-level optimisation; you get it through PyTorch's `scaled_dot_product_attention` or serving engines. → *More:* §3.4 Cost, §11.6 PagedAttention, vLLM and serving engines

Q42. Roughly how many parameters does one transformer block have? About **$12 \cdot d^2$** for model width d: attention has four d×d matrices (Q, K, V, output) = $4d^2$, and a standard FFN with 4× expansion has two d×4d matrices = $8d^2$. Multiply by the number of layers and add the embedding matrix (vocabulary × d). For example, d = 4096 and 32 layers gives about $12 \times 4096^2 \times 32 \approx 6.4$B plus embeddings — close to a "7B" model. → *More:* §3.6 From-scratch PyTorch block

4. Pre-training, post-training and fine-tuning

Q43. What is the difference between pre-training and fine-tuning? Pre-training teaches general language and world knowledge from scratch: next-token prediction over trillions of unlabelled tokens, costing millions of dollars of GPU time. **Fine-tuning** starts from those weights and continues training on a much smaller, task- or behaviour-specific dataset (thousands to hundreds of thousands of examples) to change *how* the model behaves — format, tone, domain style, a narrow skill. Fine-tuning is cheap by comparison and is mostly about behaviour, not about adding large amounts of new knowledge. → *More:* §4.3 Pretraining, §8.2 Fine-tuning

Q44. What data are LLMs pre-trained on, and what goes into preparing it? Mostly filtered web crawl (Common Crawl), plus code, books, academic papers, Wikipedia and increasingly synthetic data, totalling trillions of tokens. Preparation: language identification, quality filtering (heuristics and classifiers), heavy **deduplication** (repeated text is memorised and wastes compute), removal of personal data and toxic content, decontamination against evaluation benchmarks, and choosing the **data mix** (how much code vs web vs maths). Data quality now matters as much as model size. → *More:* §4.3 Pretraining: data and scale

Q45. What is instruction tuning (SFT), and what problem does it solve? Supervised fine-tuning trains the base model on (instruction, ideal response) pairs written or checked by people, with the loss computed only on the response tokens. It turns a text continuer into an assistant that answers questions, follows formats and uses the chat template. A few tens of thousands of high-quality examples go a long way; quality and diversity matter more than quantity. → *More:* §4.5 Post-training I: instruction tuning

Q46. ⋆ What is RLHF? Explain how it works. Reinforcement Learning from Human Feedback (popularised by InstructGPT, 2022) has three steps: (1) SFT; (2) collect human **preferences** — people rank several model answers to the same prompt — and train a **reward model** that scores an answer; (3) optimise the LLM with reinforcement learning (usually PPO, proximal policy optimisation) to maximise the reward, with a **KL penalty** that keeps it close to the SFT model so it does not drift into gibberish that fools the reward model. It aligns the model to what people prefer — helpful, honest, harmless — which is hard to write down as SFT examples. → *More:* §4.6 How RLHF works

Q47. ⋆ What is DPO, and how does it differ from RLHF? Direct Preference Optimization (Rafailov et al., 2023) skips the separate reward model and the RL loop. Using the same (prompt, chosen, rejected) pairs, it applies a simple classification-style loss that raises the probability of the chosen answer relative to the rejected one, measured against a frozen reference model; β controls how far it may move from the reference. It is simpler, more stable and cheaper than PPO-based RLHF; RL methods can still win when you have a good reward signal and want online exploration.

```
import torch.nn.functional as F
def dpo_loss(pi_c, pi_r, ref_c, ref_r, beta=0.1):   # summed log-probs of chosen/rejected
    margin = (pi_c - ref_c) - (pi_r - ref_r)
    return -F.logsigmoid(beta * margin).mean()
```

→ *More:* §4.6 DPO

Q48. What is reward hacking in RLHF? The policy finds outputs that score highly with the reward model without actually being better — for example, longer answers, confident tone, flattery (sycophancy) or repeating the question — because the reward model is only an imperfect proxy for human judgement. Defences: the KL penalty, better and more varied preference data, length-normalised or ensemble reward models, regular human review of high-reward samples, and verifiable rewards where possible. → *More:* §4.6 Common mistakes, §12.8 Alignment basics

Q49. Explain the main preference-alignment methods and their trade-offs (PPO, DPO, KTO, ORPO, GRPO, RLAIF). PPO-RLHF: reward model + RL; powerful but complex and unstable. **DPO**: direct loss on preference pairs; simple, offline. **KTO**: needs only thumbs-up/thumbs-down labels per answer rather than pairs. **ORPO**: folds preference learning into SFT in one stage with no reference model. **GRPO** (DeepSeek): RL that scores a group of sampled answers against each other instead of training a value network, widely used for reasoning with verifiable rewards. **RLAIF / Constitutional AI** (Anthropic): an AI model, guided by written principles, produces the preference labels, which cuts human labelling cost. → *More:* §4.6 Other variants, §12.8 Alignment basics

Q50. What is RL from verifiable rewards (RLVR)? Instead of a learned reward model, the reward comes from a checker: the maths answer matches, the code passes its unit tests, the output parses against a schema. Because the reward cannot be fooled the way a learned reward model can, the model can be trained for long, and it learns to produce useful chains of reasoning. This is the main recipe behind recent reasoning models. → *More:* §4.6 RL from verifiable rewards, §4.11 Reasoning models

Q51. ⋆ When should you use prompt engineering vs RAG vs fine-tuning? Use the cheapest tool that fixes the actual gap. **Prompting** first — instructions, examples and format — because it is instant and free to change. **RAG** when the gap is *knowledge*: private, large, changing or citable facts. **Fine-tuning** when the gap is *behaviour*: a consistent format or style, a narrow task at high volume, or distilling a big model's skill into a small, cheap one. They combine: many production systems fine-tune a small model *and* retrieve facts at request time. → *More:* §8.1 The adaptation question, §15.2 Decision cheat sheets

Q52. Can you teach an LLM new facts by fine-tuning? What are the limits of fine-tuning compared with RAG? Partly, but poorly: facts seen a few times in fine-tuning are recalled unreliably, the model may hallucinate *more* confidently around them, there is no citation to show where an answer came from, updates require retraining, and you cannot delete one document or enforce per-user permissions. RAG

keeps knowledge in a store you can update, filter by permission and cite. RAG's limits: it depends on retrieval quality, adds latency and tokens, and cannot change style or skill. → *More:* §8.1 The adaptation question

Q53. What is parameter-efficient fine-tuning (PEFT), and what problem does it solve? Full fine-tuning updates every weight: for a 7B model with the Adam optimiser in mixed precision that is roughly 16 bytes per parameter (≈112 GB) of GPU memory, plus a full model copy per task. PEFT freezes the base model and trains a small number of extra parameters (often under 1%), cutting memory, allowing small per-task files and reducing **catastrophic forgetting**. Families: **additive** (adapters, prefix/prompt tuning), **reparameterised** (LoRA and variants) and **selective** (train only some layers or biases, such as BitFit). → *More:* §8.2 Fine-tuning: full vs parameter-efficient

Q54. ⋆ What is LoRA, and how does it work? Low-Rank Adaptation freezes a weight matrix W (d × k) and learns an update as the product of two thin matrices, ΔW = B·A, with A of shape r × k and B of shape d × r, where the rank r is small (4–64). The layer computes W·x + (α/r)·B·A·x; B starts at zero so training begins exactly at the base model. For d = k = 4096 and r = 8 that is 65K trainable parameters instead of 16.8M. After training, B·A can be merged into W, so inference is no slower.

```
import torch, torch.nn as nn
class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r=8, alpha=16):
        super().__init__(); self.base = base.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(r, base.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r))
        self.scale = alpha / r
    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale
```

→ *More:* §8.2 LoRA

Q55. Why does LoRA work? What is the key idea behind it? The hypothesis (supported by experiments) is that the weight *change* needed to adapt a pre-trained model to a task has low **intrinsic rank** — it lives in a small subspace — so a rank-8 or rank-16 update captures most of what full fine-tuning would learn. The pre-trained weights already contain the general skills; the task only needs a small steer. → *More:* §8.2 LoRA

Q56. ⋆ What is QLoRA, and when would you use it instead of LoRA? QLoRA loads the frozen base model in **4-bit** precision (the NF4 "normal float" format, with double quantisation of the scaling constants) and trains LoRA adapters in 16-bit on top, using paged optimiser states to survive memory spikes. It cuts memory by roughly 4× versus a 16-bit base, so a 65–70B model can be fine-tuned on a single 48–80 GB GPU. Use it when GPU memory is the constraint; plain LoRA is a little faster and avoids quantisation error when memory allows. → *More:* §8.2 QLoRA

Q57. Which LoRA hyperparameters matter, and which layers do you apply it to? Rank r (capacity; 8–16 is a common start, higher for harder tasks), **alpha** (scales the update; often 1–2× r), **target modules** (attention projections q/k/v/o; adding the FFN layers usually helps), learning rate (higher than full fine-tuning, around 1e-4 to 2e-4), and epochs (1–3; more overfits). Tune on a held-out evaluation set, not on training loss. → *More:* §8.2 Fine-tuning

Q58. What is catastrophic forgetting, and how do you prevent it? Fine-tuning on a narrow dataset can overwrite general abilities: the model gets better at your task but worse at reasoning, other languages or safety behaviour. Prevent it with PEFT (the base weights stay frozen), a low learning rate and few epochs, mixing some general instruction data into the training set (replay), and evaluating on a general benchmark set alongside your task set before shipping. → *More:* §8.2 Fine-tuning

Q59. How do you prevent overfitting during fine-tuning? Use a held-out validation set and stop early when validation loss or task metrics stop improving; train for 1–3 epochs; use PEFT, which has few parameters to overfit; deduplicate and diversify the data; add regularisation (dropout in LoRA layers, weight decay); and judge by real task evaluations — generation quality — because training loss alone hides overfitting. → *More:* §8.3 Tuning data

Q60. How do you create a fine-tuning dataset? How much data do you need? Collect real inputs from your product, write or generate ideal outputs, and have experts review them; format them as chat messages in the model's template. Cover the real distribution, including edge cases and refusals, and remove

duplicates and personal data. A few hundred excellent examples can change format and style; a few thousand to tens of thousands are typical for a narrow skill. **Synthetic data** from a stronger model ("distillation") is common — filter it hard and check the provider's terms. → *More:* §8.3 Tuning data: instructions, preferences and distillation

Q61. How would you fine-tune an LLM on consumer hardware with limited GPU memory? Combine memory savers: QLoRA (4-bit frozen base, small adapters), **gradient checkpointing** (recompute activations in the backward pass instead of storing them), small micro-batches with **gradient accumulation**, a shorter maximum sequence length, mixed precision (bf16), a paged or 8-bit optimiser, and a smaller base model (7–8B). Tools such as Hugging Face PEFT/TRL or Unsloth package these. → *More:* §8.2 Fine-tuning

Q62. What is gradient accumulation? What is mixed-precision training? Gradient accumulation runs several small micro-batches, adds up their gradients, and only then updates the weights, so you get the effect of a large batch without the memory. **Mixed precision** does most maths in 16-bit (bf16 or fp16) for speed and memory, while keeping a 32-bit master copy of weights and optimiser state for stable updates; bf16 is preferred because its wide exponent range rarely overflows. Low-precision formats such as **FP8** push this further on newer GPUs. → *More:* §11.7 Quantization (precision formats)

5. Decoding, context windows and hallucination

Q63. What is autoregressive generation? What are its strengths and limits? The model generates one token at a time, each conditioned on everything before it, and appends it to the input for the next step. Strengths: one simple training objective, works for any text task, supports streaming. Limits: generation is sequential, so long outputs are slow (latency grows with output length); an early mistake cannot be revised; and each token needs a pass through the whole model, which is why serving is memory-bandwidth bound. → *More:* §4.2 Next-token prediction, §4.9 Inference

Q64. ⋆ What are the different decoding strategies in LLMs? Greedy: always take the most likely token — deterministic but repetitive and can miss better sequences. **Beam search**: keep the B best partial sequences — good for translation and other tasks with one right answer, bland for open-ended text. **Sampling**: draw from the probability distribution, shaped by **temperature**, **top-k** (only the k most likely tokens) and **top-p / nucleus** (the smallest set whose probabilities add up to p). Plus controls: stop sequences, max tokens, repetition/frequency/presence penalties, and constrained decoding for structured output. → *More:* §4.7 Decoding strategies

Q65. ⋆ What does temperature do? How do you set it? Temperature T divides the logits before softmax: softmax(z / T). T < 1 sharpens the distribution (more predictable), T > 1 flattens it (more varied and more errors), and T → 0 approaches greedy decoding. Use roughly 0–0.3 for extraction, classification and code; 0.7–1.0 for brainstorming and creative writing. Note that some newer reasoning models reject temperature and similar settings and are steered with an effort or thinking-budget parameter instead. → *More:* §4.7 Decoding strategies, §6.2 Calling an LLM API

Q66. ⋆ Explain top-k vs top-p (nucleus) sampling. Implement top-p. Top-k keeps a fixed number of candidates, which is too many when the model is confident and too few when it is unsure. Top-p adapts: it keeps the smallest set of tokens whose cumulative probability reaches p (for example 0.9), so the set is small when one token dominates and large when many are plausible. Providers usually advise tuning temperature *or* top-p, not both.

```python
import numpy as np
def top_p_sample(probs, p=0.9, rng=np.random.default_rng()):
    order = np.argsort(probs)[::-1]
    cum = np.cumsum(probs[order])
    keep = order[: np.searchsorted(cum, p) + 1]      # smallest set reaching p
    q = probs[keep] / probs[keep].sum()
    return int(rng.choice(keep, p=q))
```

→ *More:* §4.7 Decoding strategies

Q67. If I set temperature to 0, is the output deterministic? Not guaranteed. Temperature 0 means greedy decoding in principle, but servers batch many requests together, and the floating-point results of GPU kernels can differ slightly with batch composition, so near-ties can flip and the text diverges from there. Some providers also change model snapshots behind an alias. For reproducibility, pin the model

version, log the prompt and outputs, cache responses, and design evaluations to tolerate small variation. → *More:* §4.7 Common mistakes, §6.2 Calling an LLM API

Q68. When is beam search better than sampling? How is it different from BFS/DFS? Beam search suits tasks with a single correct output — translation, speech recognition, short structured answers — where you want the most probable sequence. For open-ended chat it produces generic, repetitive text, so sampling is used. Unlike BFS (expands every node at a depth) or DFS (one path to the end), beam search keeps only the B highest-scoring partial sequences at each step — a pruned breadth-first search that is not guaranteed to find the best sequence. → *More:* §4.7 Decoding strategies, §14.9 Case: machine translation

Q69. What are stop sequences and other stopping criteria? Generation stops when the model emits its end-of-sequence token, when it reaches the **max tokens** limit (the response is then cut off — check the API's stop reason), or when it produces a **stop sequence** you specified (for example "\nUser:" or a closing tag). Applications should always check *why* generation stopped, since a truncated JSON object will not parse. → *More:* §4.7 Other controls you will see in APIs, §4.9 Streaming and output limits

Q70. What are frequency and presence penalties? Both lower the logits of tokens that have already appeared. A **frequency penalty** grows with how many times the token appeared (reduces word-for-word repetition); a **presence penalty** is a flat penalty once a token has appeared at all (encourages new topics). Use small values; large ones damage fluency and can break code or JSON, where repeating tokens is correct. → *More:* §4.7 Other controls you will see in APIs

Q71. ⋆ What is a context window? What are the pros and cons of large vs small context windows? The context window is the maximum number of tokens (input plus output) the model can process in one call — everything it can "see". Large windows (hundreds of thousands to over a million tokens) let you pass whole documents or codebases. Costs: you pay for every token on every call, latency (time to first token) grows, and quality often drops for facts buried in the middle ("lost in the middle"). Smaller, well-chosen context is usually more accurate and far cheaper — which is why RAG and summarisation still matter. → *More:* §4.8 Context windows and statelessness

Q72. LLMs are stateless — so how does a chatbot remember the conversation? The model keeps nothing between calls. The application re-sends the conversation history (or a summary of it) with every request. As the history grows you manage it: keep the last N turns, summarise older turns, or store facts in long-term memory and retrieve relevant ones. Provider-side prompt caching makes re-sending a long shared prefix cheaper, but it is still re-sent. → *More:* §4.8 Stateless by default, §6.9 Conversation memory management

Q73. What is the "lost in the middle" problem? Models recall information placed at the start or end of a long context better than information in the middle (Liu et al., 2023). Newer models are better but not immune, especially when they must combine several facts. Practical response: retrieve fewer, more relevant chunks; put the most important material and the question near the end (or both ends); and test long-context recall on your own data. → *More:* §4.8 Long context is not a strategy by itself

Q74. ⋆ What is hallucination in LLMs, and why does it happen? A hallucination is fluent, confident output that is false or not supported by the given sources — an invented citation, API, fact or number. It happens because the model is trained to produce *plausible* continuations, not to check truth; when knowledge is missing, outdated, rare or private, the most plausible-sounding continuation is often wrong. Training that rewards guessing over saying "I don't know" makes it worse, and decoding randomness adds variation. → *More:* §12.2 Hallucination, §4.10 Capabilities and limits

Q75. What are the different types of hallucination? Intrinsic: contradicts the provided source (the document says 2019, the summary says 2021). **Extrinsic**: adds claims that the source does not contain and cannot be checked against it. Another split: **factuality** errors (wrong about the world) vs **faithfulness** errors (not supported by the given context). In RAG systems, faithfulness to the retrieved context is what you measure and enforce. → *More:* §12.2 Hallucination, §7.8 Keeping answers grounded

Q76. ⋆ How do you reduce hallucinations? Give techniques at each level. Data/knowledge: ground answers with RAG or tool calls (search, database, calculator). **Prompt**: tell the model to answer only from the provided sources, cite them, and say "I don't know" when they are insufficient; lower the temperature for factual tasks. **Output**: verify claims against sources (an NLI model or an LLM judge), validate structured output, and use self-consistency (sample several answers and check agreement). **Product**: show citations, keep a human in the loop for high-stakes outputs, and monitor with evaluations. You cannot

reach zero; you design so that errors are caught or are cheap. → *More:* §12.2 Hallucination, §7.8 Keeping answers grounded

Q77. What is Chain-of-Verification (CoVe)? A prompting pattern for checking facts: (1) draft an answer; (2) have the model write verification questions about each claim; (3) answer those questions independently, without showing it the draft, so it does not just repeat its errors; (4) write a final answer that keeps only the claims that survived. It reduces hallucinated facts in list-style and long-form answers at the cost of several extra calls. → *More:* §12.2 Hallucination (verification), §5.9 Prompt chaining

6. Prompt engineering

Q78. What is prompt engineering, and why is it important? Prompt engineering is designing the input to a model — instructions, context, examples and output format — so it reliably produces what you need. It matters because it is the cheapest, fastest lever you have: no training, changes in minutes, and large quality differences. Treat prompts like code: version them, test them against an evaluation set, and review changes. → *More:* §5.1 What prompt engineering is

Q79. What are the elements of a good prompt? A clear **task** (what to do), the **context** it needs (documents, data, who the user is), **constraints** (audience, length, tone, what not to do), the **output format** (a schema, headings, JSON), and **examples** when the format or judgement is subtle. Separate instructions from data with delimiters, explain *why* a rule exists, and say what to do in edge cases ("if the answer is not in the documents, say so"). → *More:* §5.2 Anatomy of a good prompt

Q80. ⋆ What is the difference between zero-shot and few-shot prompting? What is in-context learning? **Zero-shot**: instructions only, no examples. **Few-shot**: include a few input→output examples in the prompt so the model copies the pattern. **In-context learning (ICL)** is the general ability that makes few-shot work: the model adapts to a task from examples in its context *without any weight updates*. Use few-shot when format or style is hard to describe; it costs extra tokens on every call. → *More:* §5.3 Zero-shot and few-shot prompting

Q81. How do you choose examples for few-shot prompting? When does in-context learning fail? Pick examples that are representative of real inputs, diverse (cover the main cases and at least one edge case), correct, consistently formatted, and balanced across labels so the model does not copy a majority label. You can also retrieve the most similar examples for each query from an example bank (dynamic few-shot). ICL struggles when the task needs knowledge the model lacks, when examples are inconsistent, when the order or label balance biases it, or when the task is too complex to infer from a few cases — then fine-tuning or a better model is needed. → *More:* §5.3 Zero-shot and few-shot prompting

Q82. What is a system prompt, and how is it different from a user prompt? The system prompt is set by the developer and applies to the whole conversation: role, rules, tone, tools, output format and safety policy. User prompts are the end user's turns. Models are trained to give system instructions higher priority, but this is a soft preference, not a security boundary — a determined user can still try to override it, so enforce real permissions in code. → *More:* §5.4 Role / system prompts

Q83. ⋆ What is chain-of-thought (CoT) prompting? Why does it work, and what does it cost? CoT asks the model to reason step by step before answering ("think through this, then give the final answer"), or shows worked examples that do so. It works because each generated token is extra computation: writing intermediate steps lets the model break a multi-step problem into easy steps it can condition on. Costs: more output tokens (cost and latency), and the written reasoning may not faithfully reflect how the answer was produced. With reasoning models, which think internally, explicit CoT prompts help less. → *More:* §5.7 Chain-of-thought and reasoning prompts

Q84. What is self-consistency prompting? Sample several chain-of-thought answers at a non-zero temperature and take the majority final answer. Different reasoning paths make different mistakes, so voting cancels many of them. It works best when the answer is short and checkable (a number, a label); it multiplies cost by the number of samples. The level of agreement also works as a confidence signal. → *More:* §5.8 Self-consistency

Q85. Your chain-of-thought prompt still fails. How else can you improve reasoning? Break the task into a chain of smaller prompts; give the model tools (a code interpreter for maths, search for facts); add

worked examples of the hard step; use self-consistency or a verifier that checks the answer; try "tree of thoughts"-style search over multiple branches; switch to a reasoning model or raise its effort setting; and if the task is repeated at volume, fine-tune on correct reasoning traces. → *More:* §5.9 Prompt chaining, §4.11 Reasoning models

Q86. What is prompt chaining, and when is it better than one big prompt? Prompt chaining splits a task into steps where each LLM call's output feeds the next (for example extract → classify → draft → check). Each step is simpler and more reliable, can use a different model or temperature, and can be tested and debugged on its own; code can validate between steps. The cost is more calls and latency, and errors can carry forward unless each step is checked. → *More:* §5.9 Prompt chaining

Q87. ⋆ How would you get an LLM to reliably output valid JSON? Three levels of reliability: (1) prompt for JSON with a schema and an example, then parse and **validate** (for example with Pydantic) and retry with the error message on failure; (2) use the provider's **JSON mode**; (3) best, use **native structured outputs / constrained decoding**, where the server only lets the model pick tokens that keep the output valid against your JSON Schema, so it always parses. Even then, validate the *values* (ranges, business rules), because valid JSON can still be wrong. → *More:* §5.6 Structured output, §6.4 Structured output (JSON mode and schemas)

Q88. Why use delimiters or XML tags in prompts? They mark clearly where instructions end and data begins (`<document>…</document>`, `<email>…</email>`), so the model does not confuse pasted content with instructions, and you can refer to sections by name ("summarise the text in "). They also make output easy to parse. They reduce, but do not prevent, prompt injection. → *More:* §5.5 Delimiters and XML tags

Q89. What is ReAct prompting? ReAct (Reason + Act, Yao et al., 2022) interleaves **thoughts** (reasoning about what to do next), **actions** (tool calls, such as a search) and **observations** (the tool results) in a loop until the model can answer. It grounds reasoning in real data and makes the steps inspectable. It is the basis of most agent loops; today it is usually implemented with native tool calling rather than parsing "Action:" text. → *More:* §5.10 ReAct preview, §9.2 The agent loop and ReAct

Q90. How do you evaluate and iterate on a prompt? Build a test set of real inputs with expected outputs or grading criteria (including edge cases), run every prompt version against it, and score with code checks (exact match, schema validity) plus an LLM judge or human review for open-ended quality. Change one thing at a time, keep prompts in version control, and re-run the full set before shipping — a fix for one case often breaks another. → *More:* §5.14 Evaluating prompts, §5.12 The iterative refinement loop

Q91. What are common prompt pitfalls, and how do you fix them? Vague instructions (state the task, audience and format), too many competing rules (prioritise and explain why), negative-only instructions ("don't be verbose" → "answer in 3 bullet points"), examples that contradict instructions, missing context the model cannot guess, and asking for several tasks in one call (chain them). Also: not reading real outputs. Inspect failures and tie each prompt change to a test case. → *More:* §5.13 Troubleshooting prompt pitfalls

7. Building LLM applications

Q92. Walk me through the steps of running an inference request against an LLM API. The app builds the request (model, system prompt, message history, tools, max tokens, sampling settings); the provider tokenizes it and runs **prefill** (processes all input tokens in parallel, filling the KV cache — this sets the time to first token), then **decode** (generates output tokens one at a time, streaming them if requested) until a stop condition. The response comes back with the content, a stop reason and token usage, which the app logs for cost, validates and shows to the user. → *More:* §6.2 Calling an LLM API, §4.9 Inference: prefill, decode and the KV cache

Q93. What is streaming, and why do chat products use it? With streaming the server sends tokens as they are generated (typically using server-sent events, SSE, a one-way HTTP stream) instead of waiting for the whole answer. Total time is the same, but perceived latency drops to the time to first token, so users start reading in under a second. Costs: harder error handling mid-stream, output guardrails must work on partial text or buffer, and structured output can only be parsed at the end (or incrementally). → *More:* §6.3 Streaming, §11.12 Streaming responses

Q94. ⋆ What is function calling (tool use), and how does the loop work end to end? You describe tools to the model (name, description, JSON Schema for arguments). When a tool would help, the model replies with a structured **tool call** instead of text; *your code* executes the function (the model never runs anything itself), sends the result back as a tool-result message, and calls the model again; this repeats until the model gives a final answer. Validate arguments, check permissions, and cap the number of iterations.

```
messages = [{"role": "user", "content": question}]
for _ in range(MAX_STEPS):
    resp = llm(messages, tools=TOOLS)
    if resp.stop_reason != "tool_use":
        break                                   # final answer
    messages.append({"role": "assistant", "content": resp.content})
    results = [run_tool(c.name, c.input, c.id) for c in resp.tool_calls]
    messages.append({"role": "user", "content": results})
```

→ *More:* §6.5 Tool use (function calling), §9.3 Tool (function) calling

Q95. ⋆ How do you estimate the cost of an LLM feature? API vs self-hosted? API cost per request = input tokens × input price + output tokens × output price (output usually costs several times more per token); multiply by requests per day, and add retries, RAG context, tool loops and judge calls. Self-hosted cost = GPU-hours (including idle capacity and redundancy) ÷ tokens served, plus engineering and operations. Self-hosting only wins at high, steady utilisation or when data rules demand it. Always estimate with real token counts from logs, not guesses. → *More:* §6.6 Tokens, counting and cost estimation, §11.15 Cost estimation, §13.9 Cost modelling

Q96. How do you handle rate limits, timeouts and transient API errors? Retry only retryable errors (429 rate-limited, 5xx, timeouts, overloaded) with **exponential backoff plus jitter** (random spread so clients do not retry in sync), honour any `retry-after` header, and cap total attempts and time. Set timeouts on every call; make side-effecting steps idempotent (safe to repeat); use a queue and concurrency limits for batch work; and have a fallback model or provider for outages. → *More:* §6.7 Retries, rate limits and timeouts, §11.16 Rate limiting, backpressure and provider risk

Q97. ⋆ What is prompt caching, and how is it different from response (semantic) caching? Prompt (prefix) caching is done by the provider or serving engine: if a request starts with the same long prefix as a recent one (system prompt, tool definitions, a document), the stored KV cache for that prefix is reused, so those input tokens are billed at a steep discount and time to first token drops. Put stable content first and variable content last to benefit. **Response caching** is done by your app: return a stored answer for an identical request (exact cache) or a sufficiently similar one by embedding similarity (semantic cache) — no model call at all, but with a risk of returning a stale or wrong answer for a subtly different question. → *More:* §6.8 Caching, §11.13 Caching: exact, semantic and prefix

Q98. How do you manage conversation memory in a long chat? Short-term: a sliding window of recent turns, plus a running summary of older turns, trimmed to a token budget. Long-term: extract durable facts and preferences, store them (often in a vector store or a database keyed by user) and retrieve the relevant ones per request. Mind privacy — let users see and delete memories — and keep the system prompt and key instructions from being trimmed away. → *More:* §6.9 Conversation memory management, §9.6 Memory

Q99. What LLM framework do you use (LangChain, LlamaIndex, …)? Are frameworks necessary? A balanced answer: frameworks (LangChain/LangGraph, LlamaIndex, Haystack, the provider agent SDKs) speed up prototypes with ready-made loaders, retrievers and agent loops, but they hide the actual prompts and add abstraction layers that make debugging harder. Many teams start with a framework and move core paths to plain SDK calls they fully control. Whatever you use, you must be able to see the exact final prompt and every tool call. → *More:* §6.1 The landscape of modern AI tools

Q100. How do you count tokens before sending a request? Use the model's own tokenizer (for example tiktoken for OpenAI models) or the provider's token-counting endpoint (Anthropic offers one), because different model families split text differently. For rough planning, about 4 characters or ¾ of a word per English token. Count the whole request — system prompt, history, tool schemas, retrieved documents — and reserve room for the output. → *More:* §6.6 Tokens, counting and cost estimation

Q101. What is a model gateway (LLM proxy), and why use one? A gateway is a service between your apps and model providers that gives one API to many models. It centralises keys and access control, per-team budgets and rate limits, retries and fallback between providers, caching, logging and tracing, PII redaction and guardrails, and routing to cheaper models. It turns "which model?" into configuration rather than code. → *More:* §13.7 The GenAI product reference architecture, §11.14 Routing and cascades

Q102. What is LLMOps? How do you safely change a prompt or model in production? LLMOps is the operational practice around LLM features: version prompts and model IDs like code, keep a golden evaluation set, run it as a regression test in CI on every change, roll out with canaries or A/B tests, monitor quality, cost, latency and safety signals, and keep traces so failures can be replayed. A model upgrade is a release: re-run evaluations, compare, then roll out gradually with a rollback path. → *More:* §13.8 LLMOps

8. Retrieval-augmented generation (RAG)

Q103. ⋆ What is RAG, and how does it work? Retrieval-Augmented Generation answers a question in two steps: **retrieve** the passages most relevant to the question from your own knowledge base (usually by embedding similarity, often combined with keyword search), then **generate** an answer with the LLM, with those passages placed in the prompt and an instruction to answer only from them and cite them. It is really two systems: an offline **ingestion** pipeline (parse, chunk, embed, index) and an online **request path** (rewrite query, retrieve, rerank, assemble prompt, generate, check). → *More:* §7.1 What RAG is, and why it is two systems, §7.6 The RAG request path, end to end

Q104. What are the benefits of RAG? Why not just put everything in the context window? Benefits: answers use private and up-to-date knowledge without retraining; sources can be cited and checked; access control and deletion are enforced at retrieval time; and hallucination drops because the model works from evidence. Putting everything in a long context instead costs far more per call (you pay for all those tokens every time), is slower, often recalls worse (lost in the middle), and does not scale past the window. Long context complements RAG — for example, retrieve whole documents instead of tiny chunks. → *More:* §7.1 What RAG is, §4.8 Long context is not a strategy by itself

Q105. ⋆ Walk me through building a production-grade RAG system, component by component. Ingestion: connectors pull documents with their permissions; parse (PDF, HTML, tables); clean; chunk with overlap and structure; attach metadata (source, date, owner, access list); embed; write to a vector index plus a keyword index; keep it in sync (updates and deletions). **Request path**: authenticate the user; rewrite or expand the query; hybrid retrieval with permission filters; rerank; assemble a prompt with numbered sources within a token budget; generate with citations; verify grounding; return the answer with links. **Around it**: caching, guardrails, tracing, an evaluation set, and feedback collection. → *More:* §7.5 Ingestion, §7.6 The request path, §7.12 Capstone: enterprise document Q&A

Q106. What is chunking, and why do we chunk documents? Chunking splits documents into smaller passages before embedding them. Reasons: embedding models have input limits; one vector for a whole long document blurs many topics into an average that matches nothing well; and the LLM's context should receive only the relevant parts, which keeps cost down and precision up. The chunk is the unit you retrieve and cite, so its quality largely determines RAG quality. → *More:* §7.5 Ingestion: parsing, chunking, metadata and sync

Q107. ⋆ What are the different chunking strategies? Fixed-size (N tokens with some overlap, for example 500 with 50–100 overlap): simple, can cut ideas in half. **Recursive/structure-aware**: split on headings, then paragraphs, then sentences, so chunks follow the document's structure — the usual default. **Semantic**: start a new chunk where the embedding similarity between neighbouring sentences drops (a topic change). **Document-specific**: by function for code, by row group for tables, by Q&A pair for FAQs. **Parent-child / small-to-big**: retrieve small, precise chunks but send their larger parent section to the LLM. **Contextual chunking**: prepend a short summary of the document (title, section path) to each chunk before embedding. → *More:* §7.5 Ingestion

Q108. How do you choose the chunk size? What factors influence it? Small chunks are precise but lose context (a sentence without its subject); large chunks keep context but dilute the embedding and waste prompt tokens. Factors: the embedding model's input limit and the length it was trained on, the type of question (short facts vs explanations), document structure, and the LLM's context budget. Do not guess

— sweep a few sizes (for example 256, 512, 1024 tokens) and pick the best on recall@k and answer quality over your golden question set. → *More:* §7.5 Ingestion, §7.11 Evaluating RAG

Q109. Write a simple chunker with overlap. A sliding window over tokens (or words), advancing by size − overlap so neighbouring chunks share context across the cut.

```
def chunk(words, size=200, overlap=40):
    assert 0 <= overlap < size
    step = size - overlap
    return [" ".join(words[i:i + size])
            for i in range(0, max(len(words) - overlap, 1), step)]
```

In production, split on structure first (headings, paragraphs) and fall back to this only for long sections. → *More:* §7.10 A complete mini RAG pipeline

Q110. ⋆ What is hybrid search, and why use it? Hybrid search runs **keyword search** (BM25, which scores documents by matching terms, weighting rare terms more) and **vector (dense) search** in parallel and merges the results. Dense search finds paraphrases and synonyms ("car" ↔ "automobile"); keyword search nails exact terms that embeddings handle poorly — product codes, error messages, names, acronyms. Together they almost always beat either alone, so hybrid is a sensible default for enterprise RAG. → *More:* §7.7 Hybrid search and reranking

Q111. ⋆ How do you merge results from several retrievers into one ranking? (Implement Reciprocal Rank Fusion.) Scores from BM25 and cosine similarity are on different scales, so combine *ranks* instead. **Reciprocal Rank Fusion (RRF)** gives each document Σ 1/(k + rank) across the lists, with k ≈ 60; documents ranked well by several retrievers rise to the top. The alternative is to normalise scores and take a weighted sum, which needs tuning.

```
from collections import defaultdict
def rrf(rankings, k=60):                    # rankings: list of lists of doc ids
    score = defaultdict(float)
    for ranked in rankings:
        for rank, doc in enumerate(ranked, start=1):
            score[doc] += 1 / (k + rank)
    return sorted(score, key=score.get, reverse=True)
```

→ *More:* §7.7 Hybrid search and reranking

Q112. ⋆ What is reranking? Bi-encoder vs cross-encoder? A **bi-encoder** embeds the query and each document *separately*, so document vectors can be precomputed and searched fast — good for the first stage over millions of chunks, but it never looks at query and document together. A **cross-encoder** reads the query and one document *together* and outputs a relevance score — far more accurate, but too slow to run over the whole corpus. So: retrieve 50–100 candidates with a bi-encoder (plus BM25), rerank them with a cross-encoder (or an LLM), and keep the top 3–10. → *More:* §7.7 Hybrid search and reranking

Q113. How do you fine-tune a reranker? Collect (query, relevant passage) pairs from logs, clicks or human labels, add **hard negatives** (passages the current retriever ranks high but that are wrong), and train a cross-encoder with a classification or pairwise ranking loss. Evaluate with nDCG@10 or MRR on a held-out query set, and check latency, because a larger reranker adds time to every request. → *More:* §7.7 Hybrid search and reranking

Q114. What query transformations improve retrieval (query rewriting, HyDE, multi-query)? Rewriting: turn a chatty follow-up ("what about the second one?") into a standalone query using the conversation history. **Multi-query**: generate several phrasings and merge their results. **Decomposition**: split a complex question into sub-questions. **HyDE** (hypothetical document embeddings): have the LLM draft a hypothetical answer and embed *that*, since an answer looks more like the target documents than a short question does. **Step-back**: ask a more general question first. Each adds an LLM call, so measure whether recall gains justify the latency. → *More:* §7.6 The RAG request path

Q115. How do you handle multi-hop questions in RAG? Multi-hop questions need facts from several documents linked together ("Which team owns the service that caused last week's outage?"). Options: decompose into sub-questions and retrieve for each; **iterative retrieval**, where the model retrieves, reads, then issues a follow-up search (an agentic loop); or a knowledge graph of entities and relations

(**GraphRAG**) to follow links. Cap the number of hops and evaluate with multi-hop test questions. → *More:* §7.6 The RAG request path, §9.2 The agent loop

Q116. ⋆ A client's RAG system gives inaccurate answers. How do you debug it? First locate the failure using traces and a labelled question set: is the right chunk **in the index** (parsing/chunking problem), **retrieved** in the top k (retrieval problem — measure recall@k), **ranked high enough** to be sent (reranking/k problem), and **used faithfully** by the LLM (generation problem — measure faithfulness)? Then fix that stage: better parsing and chunking, hybrid search, metadata filters, a reranker, query rewriting; or for generation, a clearer grounding prompt, fewer and better chunks, or a stronger model. Change one thing at a time and re-measure. → *More:* §7.11 Evaluating RAG

Q117. How do you make RAG answers verifiable and keep them grounded? Number the retrieved passages in the prompt and require a citation for every claim; instruct the model to say it cannot answer when the sources do not support an answer; check each claim against its cited passage after generation (an NLI model or an LLM judge, checking per claim); drop or repair unsupported claims; and show the citations as links so users can check. Measure faithfulness and refusal correctness in evaluations. → *More:* §7.8 Keeping answers grounded

Q118. ⋆ How do you enforce access control in RAG so users never see documents they are not allowed to? Store each chunk with its access list (users/groups, copied from the source system at ingestion) and apply it as a **filter inside the retrieval query**, before ranking — never by asking the LLM to hide things, and preferably not by filtering after top-k (which can return empty or leak via counts). Keep permissions in sync when they change, propagate deletions quickly, separate tenants (separate indexes or strict tenant filters), and test with users who should *not* see a document. → *More:* §7.9 Access control, freshness and deletion

Q119. How do you keep a RAG index fresh, and handle document updates and deletions? Use change feeds or scheduled syncs from source systems; give each chunk a stable ID derived from document ID + position plus a content hash, so you re-embed only changed chunks and delete chunks of removed documents; store "last updated" metadata and prefer recent versions at ranking time; and track index lag as a metric. Deletion must be real (including caches and backups) for privacy rules. Changing the embedding model means re-embedding everything into a new index and switching over. → *More:* §7.9 Access control, freshness and deletion

Q120. What are advanced RAG patterns (agentic RAG, GraphRAG, corrective RAG)? Agentic RAG: the model decides when and what to retrieve, possibly across several tools and steps. **GraphRAG**: build a knowledge graph or hierarchy of community summaries from the corpus, good for "big picture" questions across many documents. **Corrective / self-RAG**: grade the retrieved passages and the draft answer, and re-retrieve or fall back to web search when they are weak. Each adds latency and cost — use them when simple RAG measurably fails on your question types. → *More:* §7.6 The RAG request path, §9.5 Planning

9. Vector databases and similarity search

Q121. ⋆ What is a vector database, and how is it different from a traditional database? A vector database stores embeddings plus metadata and answers **nearest-neighbour queries** — "the k vectors most similar to this one" — fast, using approximate indexes. A traditional database answers exact-match and range queries on structured fields using B-tree or hash indexes. Vector databases add database features on top of the index: persistence, updates and deletes, metadata filtering, replication, sharding and access control. → *More:* §7.4 Vector databases

Q122. What is the difference between a vector index, a vector database and a vector plugin/extension? A **vector index** (FAISS, HNSWlib) is a library data structure for nearest-neighbour search in memory — no persistence, no filtering, no access control. A **vector database** (Pinecone, Weaviate, Qdrant, Milvus) is a full service built around such indexes. A **vector extension** adds vector search to an existing database (pgvector for PostgreSQL, vector search in Elasticsearch/OpenSearch, Redis, MongoDB) — often the simplest choice if your data already lives there and the scale is moderate. → *More:* §7.4 Vector databases

Q123. What is approximate nearest neighbour (ANN) search, and why is it needed? Exact (brute-force) search compares the query with every vector: O(N·d) per query, too slow for tens of millions of vectors at low latency. ANN indexes trade a small amount of **recall** (occasionally missing a true nearest

neighbour) for searches that are orders of magnitude faster. You tune the trade-off with index parameters and measure recall against exact search. → *More:* §7.3 Making nearest-neighbour search fast: ANN indexes

Q124. ⋆ Explain HNSW. Hierarchical Navigable Small World builds a multi-layer graph: each vector is linked to a few close neighbours, and upper layers contain fewer nodes with long-range links. A search starts at the top layer, greedily moves to whichever neighbour is closest to the query, and drops down a layer when it can no longer improve, ending in a fine search on the bottom layer. Parameters: **M** (links per node; more = better recall, more memory), **efConstruction** (build quality) and **efSearch** (query-time breadth; more = better recall, slower). It gives excellent recall and speed but uses a lot of memory and is slower to build. → *More:* §7.3 ANN indexes

Q125. Explain IVF and product quantization (PQ). IVF (inverted file index) clusters the vectors with k-means; a query is compared with the cluster centres and only the vectors in the closest **nprobe** clusters are scanned. Higher nprobe means better recall and slower search; it fails when a true neighbour sits in a cluster that was not probed (near cluster borders). **PQ** compresses each vector by splitting it into sub-vectors and replacing each with the ID of its nearest centroid from a small codebook, cutting memory 10–60×; distances become approximate, so systems often re-rank the top candidates with full vectors. IVF-PQ combines both for billion-scale search. → *More:* §7.3 ANN indexes

Q126. What is locality-sensitive hashing (LSH)? LSH uses hash functions designed so that similar vectors are likely to get the same hash — for cosine similarity, the sign of the vector's dot product with several random directions (random projection). Only vectors in the query's bucket(s) are compared. It is simple and has theoretical guarantees, but for dense embeddings it usually needs many hash tables to reach good recall, so HNSW and IVF-PQ have largely replaced it in practice.

Q127. Which vector index would you choose for a given scenario? Small dataset (under ~100K vectors) or accuracy is the priority: **flat (exact)** search — simple and 100% recall. Millions of vectors, low latency, enough RAM: **HNSW**. Hundreds of millions to billions, or a tight memory budget: **IVF-PQ** (or a disk-based index such as DiskANN), with re-ranking of the top candidates. Frequent updates favour HNSW or flat; mostly static data tolerates IVF retraining. Always confirm recall@k vs latency on your own data. → *More:* §7.3 ANN indexes, §7.4 Vector databases

Q128. What are the challenges of metadata filtering in vector search? Post-filtering (search, then drop non-matching results) can leave fewer than k results — or none — when the filter is selective. **Pre-filtering** (filter, then search only the matching vectors) is accurate but can break graph indexes such as HNSW, whose links assume all nodes are present, and can fall back to slow brute force. Good engines filter *during* the graph traversal and switch strategy based on how selective the filter is. Permission filters must be correct, not approximate. → *More:* §7.4 Vector databases, §7.9 Access control

Q129. How do you choose a vector database? Scale (number of vectors and dimensions, growth), latency and throughput targets, need for filtering and hybrid (keyword + vector) search, update rate, multi-tenancy and access control, managed vs self-hosted, cost, and what your team already runs. A common rule: start with pgvector or your existing search engine; move to a dedicated vector database when scale or features demand it. Run a recall/latency benchmark on your data. → *More:* §7.4 Vector databases

Q130. How much memory do 10 million 1,024-dimensional embeddings need? Raw float32: 10M × 1,024 × 4 bytes ≈ **41 GB**, before index overhead (HNSW links add roughly 2·M IDs per node on the bottom layer), metadata and replicas. Ways to shrink it: float16 halves it; int8 or binary quantisation, or PQ, cuts it 4–32× (check recall); models with shorter or truncatable (Matryoshka) embeddings; or sharding across machines. → *More:* §7.3 ANN indexes

10. Evaluation (LLM-as-judge, benchmarks, RAG metrics)

Q131. ⋆ How do you evaluate an LLM application? Why is it hard? It is hard because outputs are open-ended (many correct answers), non-deterministic, and quality is multi-dimensional (correct, grounded, helpful, safe, well formatted). Build an **evaluation pyramid**: many cheap code checks (schema valid, required fields, regex, exact match), then model-graded checks (LLM-as-judge with a rubric) on a golden set of real cases, then periodic human review — plus **online** signals after launch (user feedback, edits, A/B tests). Run the offline suite on every prompt or model change. → *More:* §8.4 Why evaluating LLM systems is hard, §8.5 The evaluation pyramid

Q132. ⋆ What is LLM-as-a-judge? What are its biases, and how do you make it reliable? A strong LLM grades outputs against a rubric — either a score for one answer or a choice between two (pairwise). Known biases: **position** (prefers the first option), **verbosity** (prefers longer answers), **self-preference** (prefers its own model family) and leniency. Make it reliable by: specific rubrics with one criterion per call, asking for reasoning before the score, swapping the order in pairwise comparisons, using a different model family from the one being judged, and **calibrating against human labels** — measure agreement and fix the rubric until it is high. Re-check whenever the judge model changes. → *More:* §8.5 Offline evaluation

Q133. ⋆ How do you evaluate a RAG system? What metrics do you use? Evaluate the two halves separately. **Retrieval**: recall@k (is a relevant chunk in the top k?), precision@k, MRR and nDCG (is it near the top?) against labelled question→chunk pairs. **Generation**: **faithfulness/groundedness** (is every claim supported by the retrieved context?), **answer relevance** (does it answer the question?), correctness against a reference answer, citation accuracy, and **refusal correctness** (does it decline when the answer is not in the corpus?). Frameworks such as RAGAS automate the LLM-judged parts; calibrate them on your data. → *More:* §7.11 Evaluating RAG

Q134. Explain the common retrieval metrics: precision@k, recall@k, MRR, nDCG. When does each fail? **Precision@k**: share of the top k that are relevant. **Recall@k**: share of all relevant items that appear in the top k — the key metric for RAG, since the generator cannot use what was not retrieved. **MRR** (mean reciprocal rank): average of 1/rank of the first relevant result — good when one right answer is enough, but ignores the rest. **nDCG**: rewards relevant items more the higher they rank and supports graded relevance — best for ranked lists, but needs graded labels. All of them depend on complete relevance labels; unlabelled relevant documents make scores look worse than reality. → *More:* §7.11 Evaluating RAG, §8.6 Metrics

Q135. What are BLEU, ROUGE, BERTScore and perplexity, and what are their limits? BLEU (n-gram precision against reference texts, for translation) and **ROUGE** (n-gram recall, for summarisation) measure word overlap — cheap but blind to paraphrases and to factual errors. **BERTScore** compares contextual embeddings, so it rewards paraphrases, but still not facts. **Perplexity** is exp(average negative log-likelihood) of text under the model — lower is better language modelling, useful during training, but says little about helpfulness. For modern LLM apps these are supporting signals; task-specific checks and calibrated judges matter more. → *More:* §8.6 Metrics: what each one can and cannot tell you

Q136. ⋆ How do you choose the best LLM for your use case? Shortlist by constraints (context length, price, latency, data and hosting rules, licence), glance at public benchmarks and leaderboards, then decide on **your own evaluation set**: run each candidate on 100–500 real cases, score quality with code checks plus a calibrated judge, and record cost per request and p95 latency. Pick the cheapest model that meets the quality bar; route only hard cases to a bigger one. → *More:* §8.5 Offline evaluation, §13.6 Build vs buy

Q137. ⋆ What are the common LLM benchmarks, and why shouldn't you trust leaderboards blindly? Well-known benchmarks: **MMLU / MMLU-Pro** (multiple-choice knowledge across subjects), **GPQA** (hard graduate-level science), **GSM8K / MATH / AIME** (maths), **HumanEval / LiveCodeBench** (code), **SWE-bench** (fixing real GitHub issues), **Chatbot Arena / LMArena** (human pairwise votes), and agent benchmarks such as τ-bench. Caveats: **contamination** (test questions leak into training data), saturation (top models all score near the ceiling), benchmark-specific tuning, and a gap between benchmark tasks and your task. Use them to shortlist, never to decide. → *More:* §8.6 Metrics

Q138. How do you build a golden evaluation dataset? Sample real (anonymised) user requests across the main intents, plus edge cases, known past failures and adversarial inputs; write expected answers or grading criteria with domain experts; label difficulty and category so you can slice results; keep it versioned, with a held-out portion you never tune against; and grow it by adding every production failure you fix. A few hundred well-chosen cases beat thousands of random ones. → *More:* §8.5 Offline evaluation

Q139. What is online evaluation? How do you measure quality after launch? Offline evaluation uses fixed test sets; online evaluation measures real traffic. Signals: explicit feedback (thumbs up/down), implicit feedback (copying the answer, editing it, retrying, escalating to a human, abandoning), task success rates, and sampled LLM-judge or human review of live traces. Compare versions with A/B tests on a business metric, and watch for drift as users and data change. → *More:* §8.7 Online evaluation

Q140. How do you evaluate an AI agent? Score the **outcome** (was the task completed correctly, checked by final state — for example, did the right database row change?) and the **trajectory** (were the tool calls correct and efficient, did it ask for approval when required, did it avoid forbidden actions). Run many trials per task because agents are non-deterministic (report pass rates such as pass@k or pass^k), use sandboxed environments with simulated users and tools, and track cost, steps and latency per task. → *More:* §9.9 Evaluating agents

Q141. What are guardrail and safety evaluations? Test sets that check the system's safety behaviour: harmful requests it should refuse, borderline requests it should *not* over-refuse, prompt-injection and jailbreak attempts, PII leakage and toxic outputs. Track both false negatives (unsafe content gets through) and false positives (legitimate users blocked); run them in CI like functional tests, and add red-team findings to them. → *More:* §8.8 Guardrail evaluations and eval tooling, §12.9 Red-teaming

Q142. How do you detect regressions when you change a prompt or upgrade a model? Run the golden set on old and new versions, compare per-category scores (not just the average — a gain in one area can hide a loss in another), inspect cases that flipped from pass to fail, check cost and latency, then roll out gradually with monitoring and a quick rollback. → *More:* §13.8 LLMOps

Q143. What is pass@k? For code generation, pass@k is the probability that at least one of k sampled solutions passes the unit tests. It measures what a "generate several and test" system can achieve. Its strict cousin, pass^k (all k trials succeed), measures reliability and is used for agents, where consistency matters more than an occasional success. → *More:* §9.9 Evaluating agents

Q144. How do you measure hallucination rate in production? Sample live answers, break them into individual claims, and check each claim against the retrieved sources or a trusted reference with an NLI model or a calibrated LLM judge; report the share of unsupported claims by category. Pair this with user reports and with human review of a random sample to check the automatic checker itself. → *More:* §12.2 Hallucination, §7.8 Keeping answers grounded

11. Agents, tool calling and MCP

Q145. ⋆ What is an AI agent? How is it different from a workflow or a chatbot? An agent is an LLM running in a loop that decides *its own next step*: it picks tools, calls them, observes results, and repeats until the goal is met or a limit is hit. A **workflow** is an LLM pipeline whose steps are fixed in code (more predictable and easier to test). A chatbot just answers turn by turn. Rule: use the simplest thing that works — a single call, then a workflow, and an agent only when the path genuinely cannot be known in advance. → *More:* §9.1 What an agent is

Q146. ⋆ Explain the agent loop. What components does an agent have? Components: a **model** (the reasoning engine), **tools** (functions and APIs it can call), **instructions** (system prompt with goal and rules), **memory** (conversation state plus long-term stores) and an **orchestrator** (your code running the loop). The loop: send the goal and context → the model returns a tool call or a final answer → code checks permissions, runs the tool, appends the result → repeat, with limits on steps, time and cost, and human approval for risky actions. → *More:* §9.2 The agent loop and ReAct

Q147. How do you design good tools for an agent? Give each tool a clear name and a description written for the model (when to use it, what it returns), a strict JSON Schema for arguments, and few overlapping tools. Return concise, structured results and helpful error messages the model can act on. Make tools idempotent where possible, enforce authorisation inside the tool (not in the prompt), paginate large results, and keep the tool count small — too many similar tools confuse the model. → *More:* §9.3 Tool (function) calling

Q148. ⋆ What is the Model Context Protocol (MCP)? MCP is an open protocol (introduced by Anthropic in November 2024, donated in December 2025 to the Linux Foundation's Agentic AI Foundation, and now widely adopted) for connecting AI applications to external tools and data. A **host** (an IDE, a desktop chat app, your agent) runs one **client** per **server**; servers expose **tools** (functions the model may call), **resources** (data the app can read into context) and **prompts** (reusable templates the user picks). Messages are JSON-RPC 2.0 over **stdio** (local subprocess) or **Streamable HTTP** (remote, with OAuth-based authorisation). It turns M apps × N integrations into M + N implementations. As of the 2026-07-28 spec revision the protocol is stateless (no initialise handshake or session) — check the current spec date before quoting details. → *More:* §6.10 The Model Context Protocol, §9.8 MCP and agent protocols

Q149. What is the difference between MCP and plain function calling? Between MCP and agent-to-agent protocols? Function calling is a model-API feature: you describe tools in each request and execute them yourself. MCP standardises how those tools (and data) are *packaged and discovered*, so one server works with any MCP-capable host instead of being re-written per app; the host still turns MCP tools into function-calling definitions for the model. Agent-to-agent protocols such as A2A address a different problem — one agent delegating a task to another agent — rather than an agent using a tool. → *More:* §9.8 MCP and agent protocols

Q150. What are the security risks of MCP servers and tools? A malicious or compromised server can hide instructions in tool descriptions or results (**tool poisoning**, a form of prompt injection), change its tool definitions after you approved it, or over-request permissions; a local server runs with your user's privileges. Mitigations: install only vetted servers and pin versions, give each server least-privilege credentials, show and confirm tool calls (especially writes), treat all tool output as untrusted data, and log every call. → *More:* §6.10 The Model Context Protocol, §12.4 Prompt injection

Q151. What planning strategies do agents use (ReAct, plan-and-execute, reflection)? ReAct: think, act and observe one step at a time — flexible, but can wander. **Plan-and-execute**: first write a full plan, then execute steps (possibly with a cheaper model), re-planning when something fails — more predictable and cheaper for long tasks. **Reflection / self-critique**: the agent reviews its own output or failed attempt and tries again with the lesson. Add explicit success checks so the agent knows when it is done. → *More:* §9.5 Planning: plan-and-execute and reflection

Q152. How do agents handle memory? Short-term: the current context window (messages, tool results), trimmed or summarised as it grows. **Long-term**: facts, user preferences and past episodes saved to a store and retrieved when relevant. **Durable state**: the task's progress saved outside the model (a database or checkpoint) so long tasks survive crashes and can resume. A scratchpad or to-do file is a simple, effective form of working memory for long tasks. → *More:* §9.6 Memory: short-term, long-term and durable state

Q153. ⋆ What are multi-agent systems? When are they worth it? Several LLM agents with different roles cooperate: an **orchestrator** splits work among **sub-agents** (often in parallel, each with its own context), or agents hand off to each other (triage → billing specialist). Benefits: parallelism, smaller focused contexts and specialised prompts and tools. Costs: many more tokens, coordination failures and harder debugging. Use them when work is broad and parallel (research across many sources); one agent with good tools is better for tightly coupled tasks. → *More:* §9.7 Multi-agent patterns

Q154. ⋆ How do you put guardrails on an agent? When do you require human approval? Classify actions by risk: read-only and easily undone actions run automatically; irreversible or high-impact ones (payments, deleting data, emailing customers, deploying) need human approval. Enforce least-privilege credentials and allow-lists in code, cap steps, time, spend and retries, run code in sandboxes, validate tool arguments, and log everything for audit. Never rely on the prompt alone to stop a dangerous action. → *More:* §9.4 Guardrails, permissions and human-in-the-loop

Q155. Your agent gets stuck in a loop or keeps calling the wrong tool. How do you debug and fix it? Read the full trace (every prompt, tool call, argument and result). Common causes: vague or overlapping tool descriptions, unhelpful error messages, too many tools, results too long or truncated, missing success criteria, or context overflow that drops earlier steps. Fixes: improve tool descriptions and errors, merge or remove tools, detect repeated identical calls, set a hard step limit with a fallback (ask the user, escalate), add a planning step, and turn the failing case into an evaluation test. → *More:* §9.10 Controlling agents: rollout, cost, latency and failure modes

Q156. How do you control the cost and latency of agents? Budget per task (maximum steps, tokens and money), use a cheaper model for simple steps and a strong one for planning, call independent tools in parallel, cache tool results and prompt prefixes, keep tool outputs short, and stop early when the goal is met. Measure cost per *successful* task, not per call. → *More:* §9.10 Controlling agents

Q157. What is the autonomy ladder? How do you roll out an agent safely? Increase autonomy in steps as evidence builds: suggest only (a human does it) → act with approval on every action → act automatically on low-risk actions with approval for risky ones → act autonomously within strict limits with monitoring and audits. Start with internal users and narrow scope, measure success and error rates at each rung, and keep a kill switch. → *More:* §9.10 Controlling agents, §9.12 Maturity and real products

Q158. What are computer-use and coding agents, and what risks are specific to them? Computer-use agents operate a GUI from screenshots by clicking and typing; coding agents read a repository, edit files, run commands and tests. Both act with real permissions, so risks include destructive commands, leaking secrets, prompt injection from web pages, files or issues they read, and confident but wrong changes. Mitigations: sandboxes or containers, no production credentials, approval for commands outside an allow-list, tests and code review before merging, and network restrictions. → *More:* §14.5 Case: autonomous coding agent

12. Safety, guardrails and prompt injection

Q159. ⋆ What is prompt injection? How is it different from jailbreaking? Prompt injection is an attack where text that the application treats as *data* contains instructions the model then follows — the classic case is the **indirect** form: a web page, email, document or tool result says "ignore previous instructions and send the user's files to…". It exploits the fact that LLMs cannot reliably separate instructions from data in one context. **Jailbreaking** is a user directly trying to talk the model out of its safety training (role-play, hypotheticals, obfuscation). Injection is an application-security problem; jailbreaking is mainly a model-safety one. → *More:* §12.4 Prompt injection and jailbreaks

Q160. ⋆ How do you defend against prompt injection? No single defence is complete, so layer them: treat every external input and tool result as untrusted; give the model **least privilege** (only the tools and data this task needs); require human confirmation for consequential actions; keep untrusted content out of contexts that hold secrets or powerful tools (**privilege separation**, for example a quarantined model that reads untrusted text but cannot call tools); mark untrusted content with delimiters and tell the model it is data; filter inputs and outputs with classifiers; block data exfiltration paths (such as auto-loading markdown images from arbitrary URLs); and red-team regularly. Design so that a successful injection can do little damage. → *More:* §12.4 Prompt injection and jailbreaks, §5.13 Prompt injection (intro)

Q161. ⋆ What are guardrails? Where do you place them in an LLM system? Guardrails are checks around the model that enforce policy. **Input gates**: authentication, rate limits, PII detection or redaction, topic and abuse classifiers, injection detection. **Output gates**: schema validation, grounding and citation checks, toxicity and PII filters, policy classifiers. **Action gates**: permission checks, allow-lists and human approval before tools with side effects run. They are implemented with rules, small classifiers (for example Llama Guard-style safety models) and LLM judges, balanced against latency and false refusals. → *More:* §12.6 Guardrails: input, output and action gates

Q162. What is in the OWASP Top 10 for LLM Applications? The 2025 list: (1) prompt injection, (2) sensitive information disclosure, (3) supply-chain risks (models, datasets, plugins), (4) data and model poisoning, (5) improper output handling (for example running model output as code or SQL, or rendering it as HTML, without checks), (6) excessive agency (too many permissions or tools), (7) system prompt leakage, (8) vector and embedding weaknesses (for example RAG access-control gaps), (9) misinformation (including hallucination), (10) unbounded consumption (runaway cost or denial of service). It is a good checklist for a design review. → *More:* §12.4 Prompt injection, §12.13 A responsible-deployment checklist

Q163. ⋆ How do you protect PII and prevent data leakage in an LLM application? Minimise what you send: detect and redact or tokenise PII before the model call when it is not needed. Use providers with zero-data-retention or no-training agreements and the right data residency, or self-host. Enforce access control at retrieval so users only see their own data, separate tenants, keep secrets out of prompts (the system prompt can leak), filter outputs for PII, set retention limits and support deletion for logs, and restrict who can read traces. → *More:* §12.5 Privacy, data leakage and PII

Q164. What is "improper output handling", and why is it dangerous? Treating model output as trusted when it flows into another system: running generated SQL or shell commands, rendering generated HTML/markdown (cross-site scripting, or image links that leak data), or passing it to an API with powerful permissions. Since injected text can steer the output, this turns prompt injection into real exploits. Treat model output like user input: validate against a schema, parameterise queries, escape when rendering, sandbox execution and apply least privilege. → *More:* §12.6 Guardrails

Q165. How do bias and fairness problems show up in LLMs, and how do you test for them? Models learn stereotypes from training data, which shows up as different quality or tone across demographic groups (for example in résumé screening or loan explanations), skewed generated images, or worse performance in some languages and dialects. Test with counterfactual pairs (the same input with only a name, gender or dialect changed), slice evaluation metrics by group, and red-team sensitive use cases. Mitigate with data, prompts, output checks and — for high-stakes decisions — keep humans accountable and avoid letting the LLM make the decision alone. → *More:* §12.3 Bias and fairness

Q166. What is red-teaming for LLMs? Deliberately attacking your own system before others do: jailbreaks, prompt injections through every untrusted input, attempts to extract the system prompt or other users' data, harmful-content requests, tool misuse and cost-exhaustion attacks. Combine human experts with automated attack generation, record each successful attack as a regression test, and repeat after every major model or prompt change. → *More:* §12.9 Red-teaming and safety evaluation

Q167. What is alignment, and what is Constitutional AI? Alignment means making a model's behaviour match intended human goals and values — helpful, honest and harmless — including in situations its developers did not foresee. **Constitutional AI** (Anthropic, 2022) trains this with a written set of principles: the model critiques and revises its own outputs according to the principles, and AI feedback based on them (RLAIF) replaces much of the human preference labelling. It makes the rules explicit and scales feedback. → *More:* §12.8 Alignment basics

Q168. How do you detect AI-generated content? What are watermarking and content provenance? Text detectors (classifiers guessing whether text is AI-written) are unreliable and unfair to non-native writers — do not use them for high-stakes decisions. **Watermarking** embeds a statistical signal at generation time (for example Google's SynthID for text and images) that a detector can later find; it can be weakened by editing or paraphrasing. **Provenance** standards such as C2PA attach signed metadata ("content credentials") describing how a file was created and edited. Regulations increasingly require labelling of AI-generated media. → *More:* §12.7 Misuse: deepfakes, misinformation and detecting AI content, §10.10 Risks

Q169. What regulations and frameworks should an AI engineer know about? The **EU AI Act** (a risk-based law: prohibited practices, strict obligations for high-risk systems, transparency duties such as telling users they are talking to AI and labelling deepfakes, and rules for general-purpose AI models), the **NIST AI Risk Management Framework** (voluntary US framework: govern, map, measure, manage, with a Generative AI profile), plus data-protection law such as GDPR. Application dates have shifted; check the current timetable before quoting dates. → *More:* §12.12 Governance and regulation

Q170. A user reports that your chatbot produced harmful or false content. What do you do? Contain first (block the pattern or disable the feature if severe), then pull the trace to reproduce it, classify the root cause (prompt gap, retrieval returned a bad source, model failure, injection, missing guardrail), fix at the right layer, add the case to the evaluation and safety test sets, check logs for similar incidents, and communicate with the user and stakeholders. Treat it like any production incident, with a blameless post-mortem. → *More:* §12.13 A responsible-deployment checklist, §11.17 Observability

13. Inference serving, quantization, cost and latency

Q171. What is the difference between LLM training and inference? Training runs forward and backward passes over huge batches to update the weights, needs the weights plus gradients and optimiser state (roughly 16 bytes per parameter with Adam in mixed precision) and is measured in GPU-weeks. Inference only runs forward passes with frozen weights, needs the weights plus a KV cache, and is optimised for latency, throughput and cost per token. Over a model's life, inference usually costs more in total than training. → *More:* §11.1 The LLM as a production component

Q172. ⋆ Explain prefill vs decode. Why do they have different bottlenecks? Prefill processes all prompt tokens in parallel in one pass and fills the KV cache — lots of maths per byte of weights read, so it is **compute-bound** and determines **time to first token (TTFT)**. **Decode** generates one token at a time; each step reads all the weights and the KV cache to produce a single token per sequence, so it is **memory-bandwidth-bound** and determines **time per output token (TPOT)**. That is why batching (sharing each weight read across many sequences) raises decode throughput so much, and why some systems run prefill and decode on separate GPU pools (disaggregated serving). → *More:* §11.2 Prefill vs decode, §4.9 Inference

Q173. ⋆ How do you measure LLM inference performance? Which metrics matter? **TTFT** (time to first token — perceived responsiveness), **TPOT / inter-token latency** (streaming speed; about 20–50 ms per token reads comfortably), **end-to-end latency** ≈ TTFT + output tokens × TPOT, **throughput** (total tokens per second across users), and **goodput** (requests per second that meet the latency targets). Always report percentiles (p50, p95, p99) rather than averages, at a stated input/output length and concurrency, plus cost per million tokens. → *More:* §11.3 Latency and throughput metrics

Q174. ⋆ Explain the KV cache. How does it speed up inference, and what does it cost? In decoding, each new token must attend to all previous tokens' keys and values. Without a cache they would be recomputed at every step (quadratic work); the **KV cache** stores them, so each step computes K and V only for the new token. The cost is GPU memory: it grows with layers × KV heads × head dimension × sequence length × batch size, and for long contexts it is often larger than the weights and limits how many users fit on a GPU. → *More:* §11.4 The KV cache, §4.9 The KV cache in code

Q175. ⋆ How do you calculate the size of the KV cache? Bytes per token = 2 (K and V) × layers × KV heads × head dimension × bytes per value. For an 8B Llama-3-class model (32 layers, 8 KV heads with GQA, head dim 128, fp16): 2 × 32 × 8 × 128 × 2 = 131,072 bytes = **128 KiB per token**, so an 8K-token sequence needs 1 GiB and 32 such users need 32 GiB — on top of 16 GB of weights.

```
def kv_cache_bytes(layers, kv_heads, head_dim, tokens, batch=1, bytes_per=2):
    return 2 * layers * kv_heads * head_dim * bytes_per * tokens * batch
print(kv_cache_bytes(32, 8, 128, 8192) / 2**30)   # 1.0 GiB
```

→ *More:* §4.9 The KV cache in code, §11.10 GPU memory sizing

Q176. How do you handle the large memory requirements of the KV cache? Shrink it: GQA/MQA (fewer KV heads), KV-cache quantisation (8-bit or lower), sliding-window attention for some layers. Manage it better: **PagedAttention** (allocate in small blocks, no fragmentation), prefix sharing between requests with the same prompt start, offloading cold entries to CPU memory. Limit it: cap context length and concurrent sequences, evict or pre-empt long requests, and spread it over more GPUs with tensor parallelism. → *More:* §11.4 The KV cache, §11.6 PagedAttention, vLLM and serving engines

Q177. ⋆ What is continuous batching, and how does it differ from static batching? **Static batching** groups requests, runs them together and waits until the *longest* one finishes before starting new ones — short requests waste GPU slots and new ones queue. **Continuous (in-flight) batching** schedules at the level of each decoding step: finished sequences leave and waiting ones join immediately, so the GPU stays full. It raises throughput several-fold at similar latency and is standard in vLLM, TGI, TensorRT-LLM and SGLang. → *More:* §11.5 Batching: static, dynamic and continuous

Q178. Explain the throughput vs latency trade-off in LLM serving. Larger batches share each read of the weights across more sequences, so total tokens per second rise — but each step takes a little longer and requests may wait to join, so per-user latency rises. Interactive chat favours small batches and low TPOT; offline jobs (summarising a million documents) favour large batches and maximum throughput, often through cheaper batch APIs. Choose the operating point by your latency SLO (service-level objective) and maximise goodput within it. → *More:* §11.5 Batching, §11.3 Latency and throughput metrics

Q179. ⋆ What is PagedAttention (vLLM)? Inspired by virtual memory in operating systems, PagedAttention stores each sequence's KV cache in fixed-size **blocks** that need not be contiguous, tracked by a block table. This removes the waste of reserving memory for the maximum possible length (fragmentation), lets sequences share blocks for common prefixes (copy-on-write), and so fits far more concurrent sequences — the main reason vLLM achieved much higher throughput than earlier servers. → *More:* §11.6 PagedAttention, vLLM and serving engines

Q180. ⋆ What is quantization, and how does it affect speed, memory and accuracy? Quantization stores weights (and sometimes activations and the KV cache) in fewer bits: fp16/bf16 (2 bytes) → int8 or FP8 (1 byte) → int4 (half a byte). Memory drops 2–4×, and because decoding is memory-bandwidth-bound, fewer bytes to read means faster tokens. Accuracy loss is usually small at 8 bits and modest at 4 bits with good methods, larger for small models and for maths or code — always evaluate on your task. Methods: post-training weight-only quantisation (**GPTQ**, **AWQ**), formats for local inference (GGUF/llama.cpp), and FP8 on newer GPUs. → *More:* §11.7 Quantization

Q181. Why doesn't quantization destroy LLM accuracy? Neural network weights are highly redundant, and most weights cluster near zero, so rounding them to a 256-level (int8) or 16-level (int4) grid with a

per-group scale loses little information. The danger is a few **outlier** values — large-magnitude weights or activation channels that matter a lot; methods such as AWQ (protects the weights tied to large activations), GPTQ (corrects rounding error using second-order information) and SmoothQuant (moves outliers from activations to weights) handle them. Below about 4 bits, quality drops faster. → *More:* §11.7 Quantization

Q182. What is knowledge distillation, and why use small specialist models? Distillation trains a small **student** model to imitate a large **teacher**: on the teacher's outputs (synthetic data) or on its full probability distributions. A fine-tuned 1–8B student can match a frontier model on one narrow task at a fraction of the cost and latency, and can run on your own hardware. The trade-off: it loses generality, so route anything outside its task to a bigger model. → *More:* §11.8 Distillation and small specialist models

Q183. ⋆ What is speculative decoding, and when would you use it? A small, fast **draft** model (or extra prediction heads, as in Medusa/EAGLE) proposes the next few tokens; the large **target** model checks all of them in one parallel forward pass and accepts the longest prefix it agrees with, plus one token of its own. With the right acceptance rule the output distribution is **exactly** the target model's, so quality is unchanged. It speeds up decoding 2–3× when the draft guesses well (predictable text such as code or structured output) and spare compute is available; it helps less at large batch sizes where the GPU is already busy. → *More:* §11.9 Speculative decoding

Q184. ⋆ How much GPU memory do you need to serve a 70B model? Weights: 70B × 2 bytes (fp16) = **140 GB**; at int8 ≈ 70 GB; at int4 ≈ 35–40 GB. Add the KV cache for your concurrency and context (for a 70B GQA model about 320 KiB per token, so 2.5 GiB per 8K-token sequence) and roughly 10–20% overhead. So fp16 needs at least two 80 GB GPUs with tensor parallelism, realistically 4 for useful concurrency; int4 can fit on one 80 GB GPU with modest concurrency. → *More:* §11.10 GPU memory sizing

Q185. What are tensor parallelism and pipeline parallelism? What are the challenges of distributed inference? Tensor parallelism splits each layer's matrices across GPUs, which work on every token together and exchange results each layer — low latency but needs very fast links (NVLink), so it stays within one machine. **Pipeline parallelism** puts different layers on different GPUs, passing activations along — less communication but bubbles (idle stages) and extra latency. Training also uses data parallelism and sharded optimiser states (ZeRO/FSDP); MoE models add expert parallelism. Challenges: communication overhead, load balance, memory limits per GPU and failure handling. → *More:* §11.11 Model parallelism: tensor and pipeline

Q186. Which LLM inference engines do you know? Which would you choose? vLLM (PagedAttention, continuous batching, broad model support — a common default), **SGLang** (fast, with prefix caching via RadixAttention and structured generation), **TensorRT-LLM** (NVIDIA, highly optimised, more setup), **Hugging Face TGI**, and **llama.cpp / Ollama** for local and CPU/Mac inference. Choose by hardware, model support, features you need (LoRA serving, structured output, speculative decoding) and operational fit — then benchmark on your traffic. → *More:* §11.6 PagedAttention, vLLM and serving engines

Q187. ⋆ What are all the options for reducing LLM latency? Reduce the work: shorter prompts, fewer output tokens (ask for concise answers, set max tokens), a smaller or distilled model, routing easy requests to a fast model. Make the work faster: streaming (perceived latency), prompt/prefix caching (TTFT), quantisation, speculative decoding, FlashAttention, tensor parallelism. Avoid the work: response or semantic caching, precomputing common answers. And parallelise independent calls (retrieval, tools, sub-questions) instead of chaining them. → *More:* §11.12 Streaming responses, §11.13 Caching, §11.14 Routing and cascades

Q188. ⋆ How do you optimise the cost of an LLM system? Measure cost per request and per feature first. Then: route by difficulty (a cheap model first, escalate when confidence is low — a **cascade**), trim prompts and retrieved context, use prompt caching for long shared prefixes, cache whole responses where safe, use batch APIs for offline jobs, cap output length, distil or fine-tune a small model for high-volume narrow tasks, and self-host only when volume is high and steady. Set budgets and alerts per team or tenant. → *More:* §11.14 Routing and cascades to cheaper models, §11.15 Cost estimation

Q189. Do a back-of-the-envelope cost estimate for an LLM feature. Example: 1M requests/day, 2,000 input + 300 output tokens each. Daily tokens: 2B input + 0.3B output. At illustrative prices of $1 per million input and $5 per million output tokens: 2,000M × $1/M + 300M × $5/M = $2,000 + $1,500 = **$3,500/day ≈ $105K/month**. Then show the levers: caching a 1,500-token shared prefix at a large

discount, routing 70% of traffic to a model 10× cheaper, and trimming output. State that prices are placeholders and change often. → *More:* §11.15 Cost estimation, §13.9 Cost modelling and scale estimates, §11.18 Capstone: pricing the inference layer of a 5M-user product

Q190. What should you monitor (observe) in an LLM application in production? Per request: a trace of every step (prompt version, model, retrieved chunk IDs, tool calls, tokens, latency per span, cost, stop reason, errors). Aggregates: TTFT/latency percentiles, error and retry rates, cost per feature and tenant, cache hit rate, guardrail trigger and refusal rates, user feedback, and sampled quality scores from an LLM judge. Alert on spikes and drift; protect traces as they contain user data. → *More:* §11.17 Observability

14. Diffusion and generative models beyond text

Q191. ⋆ Compare VAEs, GANs and diffusion models. VAE (variational autoencoder): encodes data into a smooth latent distribution and decodes samples from it — stable to train and fast, but samples tend to be blurry; today mostly used as the compressor inside latent diffusion. **GAN** (generative adversarial network): a generator and a discriminator compete — sharp images and one-step (fast) sampling, but unstable training and **mode collapse** (it covers only part of the data's variety). **Diffusion**: learns to remove noise step by step — the best quality and diversity and stable training, but many sampling steps make it slower (reduced by better samplers and distillation). → *More:* §10.2 VAEs, §10.3 GANs, §10.4 Diffusion models

Q192. ⋆ How do diffusion models work? Forward process: gradually add Gaussian noise to a training image over T steps (often 1,000) until it is pure noise; a closed formula lets you jump to any step directly. **Training**: a network (a U-Net or a diffusion transformer) receives a noisy image and the step number and learns to predict the noise that was added (a simple mean-squared-error loss). **Generation**: start from random noise and repeatedly subtract the predicted noise, step by step, until an image appears. Faster samplers (DDIM and later solvers) and distilled models cut the steps from hundreds to a few dozen or fewer. → *More:* §10.4 Diffusion models

Q193. How does text-to-image generation work (CLIP, latent diffusion, classifier-free guidance)? A text encoder (CLIP's or a language model's) turns the prompt into vectors, and the denoising network attends to them through **cross-attention** at every step. **Latent diffusion** (Stable Diffusion) runs the process in a compressed latent space produced by a VAE (for example 64×64×4 instead of 512×512×3), which makes it far cheaper; the VAE decoder turns the final latent into pixels. **Classifier-free guidance** computes the noise prediction with and without the prompt and extrapolates: $\varepsilon = \varepsilon_{uncond} + w \cdot (\varepsilon_{cond} - \varepsilon_{uncond})$; a higher w follows the prompt more closely at the cost of variety and realism. → *More:* §10.5 Text-to-image: CLIP, guidance and latent diffusion

Q194. What is CLIP, and what is it used for? CLIP (OpenAI, 2021) trains an image encoder and a text encoder together with **contrastive learning** on hundreds of millions of image–caption pairs, so a matching image and caption get similar embeddings in a shared space. Uses: zero-shot image classification (compare the image with "a photo of a {label}" for each label), text-to-image search, guiding and scoring image generators (CLIP score), and as the vision encoder in multimodal models. → *More:* §10.5 Text-to-image

Q195. How do you evaluate image generation models (FID, CLIP score)? FID (Fréchet Inception Distance) compares the statistics of features from real and generated images — lower means more realistic and diverse, but it needs many samples and does not check prompt following. **CLIP score** measures how well the image matches the prompt. Human preference studies (pairwise comparisons) remain the gold standard, plus checks for specific failures (text rendering, hands, counting objects) and safety. → *More:* §10.9 Evaluating generative models

Q196. How do you personalise or control an image model (LoRA, DreamBooth, ControlNet)? DreamBooth fine-tunes the model on a few (3–20) photos of a subject tied to a rare token, so prompts can place that subject in new scenes; **LoRA** does the same cheaply with small adapter files. **ControlNet** adds a trainable copy of the encoder that conditions generation on a structure input — edges, depth, pose — for precise layout control. **Inpainting** regenerates only a masked region. Personalisation of real people needs consent and safeguards. → *More:* §14.13 Case: personalised headshot generation, §10.5 Text-to-image

Q197. How does text-to-video generation work, and why is it hard? Most current systems are diffusion transformers over **spacetime patches**: video is compressed by a video VAE into a latent tensor across frames, cut into patches, and denoised by a transformer that attends across space and time. It is hard because of temporal consistency (objects must not morph between frames), physics, enormous compute (many frames at high resolution), and scarce high-quality captioned video data. → *More:* §10.7 Video generation and deepfakes, §14.14 Case: text-to-video generation

Q198. How do speech models work (ASR and TTS)? ASR (automatic speech recognition), for example Whisper: an encoder-decoder transformer that reads a log-mel spectrogram (a picture of sound frequencies over time) and generates the transcript as text tokens. **TTS** (text-to-speech): modern systems generate discrete audio tokens with a language-model-style decoder, or a spectrogram converted to sound by a vocoder. Voice cloning from seconds of audio raises fraud risks, which calls for consent and watermarking. → *More:* §10.6 Audio and speech

Q199. What is the difference between diffusion language models and autoregressive LLMs? Autoregressive LLMs generate one token at a time, left to right. **Diffusion language models** start from a fully masked (noisy) sequence and fill in many tokens in parallel over several refinement steps, so they can be faster for long outputs and can revise earlier tokens. As of 2026 they are promising for speed-sensitive uses such as code completion, but autoregressive models still lead on quality and tooling (KV cache, streaming). (Image diffusion itself is covered in §10.4.)

15. GenAI system design and scenario questions

Q200. ⋆ How do you approach a GenAI system design interview? What framework do you use? A seven-step flow on a time budget: (1) clarify requirements and turn them into numbers (users, requests per second, latency targets, quality bar, budget, safety and privacy rules); (2) frame it as an ML/LLM task and decide what "good" means; (3) data — sources, permissions, freshness; (4) model choice — API vs open weights, prompt vs RAG vs fine-tune; (5) the end-to-end architecture; (6) evaluation offline and online; (7) deployment, serving, cost, monitoring and safety. Lead with the simplest design that works, then deepen where the interviewer pushes. → *More:* §13.4 The seven-step framework, §14.1 How to run a case study

Q201. ⋆ Design a customer-support chatbot. Requirements: answer from help-centre articles and the user's account data, resolve common tasks (order status, refunds within policy), hand off to humans, p95 first token under 1–2 s. Design: chat front end with streaming → gateway (auth, rate limits, PII redaction) → orchestrator with a small router (FAQ vs account action vs escalation) → RAG over help docs (hybrid search, reranking, citations) → tools for account lookups and refunds with policy limits and approval for anything over a threshold → output guardrails → handoff with conversation summary. Evaluate with a golden set of past tickets, resolution rate, CSAT and escalation correctness; control cost with routing and prompt caching. → *More:* §9.11 Agent system design: a customer-support agent, §13.10 Four archetype GenAI products

Q202. ⋆ Design an enterprise RAG / document Q&A system over millions of internal documents. Clarify: sources (wiki, drive, tickets), number of documents, permissions model, freshness, languages, citation requirement. Ingestion: connectors with permission sync, parsing, structure-aware chunking, metadata, embeddings, hybrid index, incremental updates and deletes. Query: auth → query rewriting → hybrid retrieval *with permission filters* → rerank → grounded generation with citations → claim verification → answer. Scale: shard the index, cache embeddings and frequent answers. Evaluate retrieval (recall@k) and generation (faithfulness, refusal correctness) separately; monitor "no answer" rates and feedback. → *More:* §7.12 Capstone: enterprise document Q&A with permissions, §14.2 Case: legal research assistant

Q203. ⋆ Design an AI coding assistant / copilot (code completion in the IDE). Two paths. **Inline completion**: very low latency (a few hundred ms), so a small, fast code model fine-tuned for fill-in-the-middle (predicting code between a prefix and a suffix), context from the current file, open tabs and nearby symbols, aggressive caching and request cancellation as the user types, speculative decoding; measure acceptance rate and retained code. **Chat/agent mode**: a strong model with repository retrieval (code search, embeddings, symbol graph), tools to read files, run tests and edit, in a sandbox with approval for commands. Plus: privacy (no training on customer code without consent), secret filtering, licence/duplication filters, and evaluation on real tasks (SWE-bench-style). → *More:* §14.5 Case: autonomous coding agent, §14.8 Case: sentence completion in email

Q204. Design ChatGPT (a general-purpose chat assistant at scale). Clarify scale (for example tens of millions of daily users), features (multi-turn chat, files, web search, memory) and latency targets. Components: clients with streaming over SSE/WebSockets → API gateway (auth, quotas, abuse) → conversation service (stores history, assembles context, summarises long chats) → safety layer (input and output classifiers) → model router (fast model by default, reasoning model for hard requests) → inference fleet (continuous batching, KV/prefix caching, autoscaling per model, multi-region) → tools (search, code execution, file retrieval). Discuss GPU capacity planning, graceful degradation under load (queueing, smaller models), cost per active user, feedback collection and evaluation. → *More:* §13.7 The GenAI product reference architecture, §11.18 Capstone: a 5M-user product

Q205. Design an AI meeting assistant (transcribe, summarise, extract action items). Pipeline: audio capture → streaming ASR with speaker diarisation (who spoke when) → transcript store → after the meeting, chunk long transcripts and summarise hierarchically (map-reduce) → extract action items into a schema (owner, task, due date) with validation → let users edit and sync to task tools. Concerns: consent and recording laws, accuracy on names and jargon (custom vocabulary), hallucinated action items (require each to cite a transcript span), retention policies, and cost per meeting hour. → *More:* §14.3 Case: AI meeting assistant

Q206. Design a text-to-image generation service at scale. Requests go through prompt safety classification → queue (generation takes seconds, so it is asynchronous with progress updates) → GPU workers running a latent diffusion model with batching and a distilled few-step variant for previews → output safety classifier, watermark and provenance metadata → object storage + CDN. Discuss GPU autoscaling on queue depth, priority tiers, caching popular prompts, cost per image, evaluation (human preference, prompt adherence, FID) and abuse (deepfakes of real people, copyrighted styles). → *More:* §14.4 Case: text-to-image product at scale

Q207. Design an LLM-powered content moderation system. Use a cascade: cheap rules and hashes for known bad content → a small fast classifier for most traffic → an LLM with the written policy for ambiguous cases → human reviewers for appeals and the hardest cases, whose decisions become training and evaluation data. Measure precision and recall per policy category, latency budgets for pre- vs post-publication checks, fairness across languages and dialects, and adversarial drift (users evading filters). → *More:* §12.6 Guardrails, §11.14 Routing and cascades

Q208. Design an AI email/autocomplete feature (smart compose). Latency is the main constraint (under ~100 ms per suggestion), so use a small model, possibly on device, conditioned on the current draft, subject and recipient; suggest only when confidence is high; personalise lightly. Train on consented email data with strong privacy (no memorisation of personal data; differential privacy or filtering). Evaluate with acceptance rate and keystrokes saved, and filter inappropriate or biased suggestions. → *More:* §14.8 Case: sentence completion in email

Q209. Design a semantic search engine for an e-commerce catalogue. Index products with text embeddings (title, description, attributes) and image embeddings, plus BM25 and structured filters (price, size, availability). Query path: query understanding (spelling, category and attribute extraction with a small model) → hybrid retrieval → learning-to-rank reranker using behavioural signals (clicks, purchases) → business rules. Evaluate offline with nDCG on labelled queries and online with click-through and conversion A/B tests. Keep the index fresh with stock and price updates. → *More:* §7.7 Hybrid search and reranking

Q210. Design a multi-tenant LLM platform (gateway) for many internal teams. One gateway API in front of several providers and self-hosted models: authentication and per-team keys, quotas and budgets, routing and fallback, prompt and response logging with PII controls, caching, guardrails as shared middleware, an evaluation service, a prompt registry with versioning, and dashboards for cost, latency and quality per team. Self-hosted models run on a shared inference cluster (vLLM, autoscaling, multi-LoRA serving for team adapters). → *More:* §13.11 Capstone — AI features for a SaaS product, §11.16 Rate limiting, backpressure and provider risk

Q211. ⋆ How would you design a scalable, low-latency LLM inference system for real-time applications? Size from the requirements: peak requests per second × tokens per request → tokens per second needed → GPUs, given measured throughput per GPU at your latency SLO. Use an engine with continuous batching and PagedAttention, prefix caching, quantisation, speculative decoding where it helps, and tensor parallelism for big models. Autoscale on queue depth and KV-cache usage (not CPU), keep warm capacity because model loading is slow, separate interactive and batch traffic, load-balance with session or

prefix affinity, and add rate limiting and graceful degradation (fallback to a smaller model) under overload. → *More:* §11.18 Capstone: pricing the inference layer of a 5M-user product, §11.16 Rate limiting, backpressure and provider risk

Q212. Design a machine translation service. Use an encoder-decoder or LLM-based model with beam search for quality; support many language pairs with one multilingual model; handle domain terminology with glossaries and constrained decoding; cache frequent segments (translation memory); stream for long documents by splitting at sentence boundaries. Evaluate with automatic metrics (BLEU, COMET) plus human review per language pair, and watch low-resource languages and gender-bias errors. → *More:* §14.9 Case: machine translation service

Q213. ⋆ Your LLM feature works in the demo but fails in production. What usually goes wrong, and how do you fix it? Typical gaps: real inputs are messier and more varied than demo inputs; there is no evaluation set, so regressions go unnoticed; retrieval misses on real queries; latency and cost explode under real traffic (long contexts, agent loops); prompt injection and abuse appear; and no tracing makes failures impossible to debug. Fix by logging and sampling real traffic into a golden set, adding automatic evaluations and guardrails, tracing every step, setting budgets and limits, and rolling out gradually. → *More:* §13.8 LLMOps, §14.6 Case: critiquing a system design

Q214. ⋆ Build vs buy: when would you use an API model, an open-weight model or train your own? API by default: best quality, fastest to ship, no infrastructure — fits most products, especially early. **Open weights, self-hosted** when data must stay inside your environment, volume is high and steady enough to beat API pricing, you need deep customisation, or you need offline/edge deployment. **Fine-tune** (either kind) for consistent behaviour or narrow high-volume tasks. **Pre-train your own** almost never — only with unique data at huge scale and a budget in the tens of millions. Keep a model abstraction layer so you can switch. → *More:* §13.6 Build vs buy

Q215. What are the non-functional requirements of a GenAI feature, and how do you turn them into numbers? Latency (TTFT and end-to-end p95), throughput (peak requests per second, tokens per second), quality (task score on the golden set above a bar, hallucination rate below a bar), cost (per request and per month, per user), availability (SLO, fallback provider), safety (refusal correctness, injection resistance), privacy and compliance (data residency, retention, no training on customer data). Writing them as numbers lets you choose models, size infrastructure and decide when you are done. → *More:* §13.5 Requirements for a GenAI feature

Sources of the question lists

Question wording was taken from these public lists and reports (titles only; every answer above is written for this book):

- KalyanKS-NLP, *LLM Interview Questions and Answers Hub* (GitHub) — github.com/KalyanKS-NLP/LLM-Interview-Questions-and-Answers-Hub
- SimranAnand1, *100+ LLM Interview Questions for Top Companies* (GitHub) — github.com/SimranAnand1/LLMInterviewQuestions
- a-tabaza, *GenAI Interview Questions* (GitHub) — github.com/a-tabaza/genai_interview_questions
- Devinterview-io, *LLMs Interview Questions* (GitHub) — github.com/Devinterview-io/llms-interview-questions
- Lau-Jonathan, *LLM-Agent-Interview-Guide* (GitHub) — github.com/Lau-Jonathan/LLM-Agent-Interview-Guide
- Praveen76, *LLMs Interview Prep Guide* (GitHub) — github.com/Praveen76/LLMs-Interview-Prep-Guide
- techinterview.org, *What applied-AI engineer interviews test in 2026* (RAG, agents, evals)
- lets-code.co.in, *Generative AI, LLM and RAG Interview Questions 2026*
- Exponent (tryexponent.com), Google AI engineer interview experience ("Design an enterprise-level RAG system")
- hirist.tech, *Top MCP interview questions: AI agents and tool calling*; claveprep.com, *Agentic AI Interview Questions 2026*
- OWASP, *Top 10 for LLM Applications (2025)* — used for the security question wording

- Chip Huyen, *AI Engineering* (O'Reilly, 2025) and this book's Chapter 13–14 interview cases — for system-design prompts